
Detección de Intrusiones en Sistemas IIoT

Iván Román Utrilla

Rafael Sánchez Navarro

Marcelo Chinarro Cabrero

Raul Martin Catalan

Prof. Julio Alberto López

Prof. José Antonio de la Torre las Heras

Introducción	2
1 Hito 1: Definición del problema y determinación del alcance	3
1.1 Definición del Problema	3
1.2 Objetivos del Proyecto	6
1.3 Determinación del Alcance	7
1.4 Conclusiones	11
2 Hito 2: Los datos: origen, descripción y análisis	13
2.1 Introducción	13
2.2 Fuentes, origen y tamaño de los datos	14
2.3 Inventario de datos	16
2.4 Análisis de los datos	23
2.5 Evaluación del rendimiento. Métricas	29
2.6 Conclusiones	33
3 Hito 3: Modelado, experimentación y validación	34
3.1 Introducción	34
3.2 Modelos	35
3.3 Configuración experimental	37
3.4 Resultados	40
3.5 Discusión de resultados	47
3.6 Interpretabilidad	50
3.7 Conclusiones	52
4 Hito 4: Despliegue y paso a producción del modelo	54
4.1 Objetivo del despliegue	54

4.2	Arquitectura y estructura del sistema	55
4.3	Entrenamiento offline e inferencia	56
4.4	Servicio de inferencia y API REST	56
4.5	Contenerización y orquestación	59
4.6	Gestión y versionado de modelos	59
4.7	Validación y pruebas del sistema	60
4.8	Limitaciones, mejoras futuras y conclusiones	61
5	Hito 5: Monitorización y ciclo de retroalimentación	63
5.1	Introducción y objetivos	63
5.2	Arquitectura de monitorización	64
5.3	Instrumentación de la API	64
5.4	Métricas operativas del servicio (DevOps)	65
5.5	Logging y trazabilidad de inferencias	65
5.6	Monitorización de predicciones (Model Health)	66
5.7	Detección de deriva de datos (Data Drift)	66
5.8	Sistema de alertas (Push Notifications)	67
5.9	Dashboard de Grafana	67
5.10	Ciclo de retroalimentación y validación humana	68
5.11	Política de reentrenamiento (<i>Retraining</i>)	69
5.12	Activación del nuevo modelo en producción	69
5.13	Prueba completa del sistema (Validación End-to-End)	69
5.14	Limitaciones y mejoras futuras	70
5.15	Conclusiones	71
A	Directorio de GitHub	72
B	Análisis Exploratorio de los datos	73
C	Visualizaciones de los resultados en entrenamiento	75
D	Visualización del proceso de monitorización	77
E	Recursos Hardware	80
	Referencias bibliográficas	83

List of Figures

2.1	Distribución de los porcentajes de ataques en el dataset X-IIoTID.	16
2.2	Distribución del tráfico en el dataset X-IIoTID. A la izquierda, la proporción casi igualitaria entre tráfico normal (51,34%) y anómalo (48,66%) en la clasificación binaria (<code>class3</code>) constituye un escenario favorable para el entrenamiento de clasificadores. A la derecha, las distribuciones por categoría (<code>class2</code>) y tipo de ataque (<code>class1</code>) muestran un marcado desequilibrio que deberá abordarse en la fase de modelado.	25
2.3	Distribución de <code>packet_rate</code> en escala logarítmica para tráfico normal (azul) y anómalo (rojo). El solapamiento entre distribuciones evidencia la insuficiencia de reglas estáticas basadas en umbrales individuales.	26
2.4	Top 15 de características con mayor porcentaje de valores nulos (véase Anexo).	28
2.5	Curva ROC que muestra la relación entre TPR y FPR.	32
3.1	Comparativa de la media de $F1$ en entrenamiento, validación y test.	41
3.2	Gap de generalización entre entrenamiento, validación y test.	42
3.3	Evolución de la pérdida y del $F1$ macro durante el entrenamiento de la DNN.	43
3.4	Curva de aprendizaje de XGBoost mediante <code>mlogloss</code> en entrenamiento y validación.	44
3.5	Evolución de la pérdida y de la puntuación de validación interna para MLP.	44
3.6	Curvas ROC binarias en test para los mejores modelos.	46
3.7	Clases más difíciles según el <i>recall</i> medio de los mejores modelos.	47
3.8	Explicación de una instancia mediante SHAP y LIME.	52
B.1	Análisis de varianza de las características.	73
B.2	Correlación de las variables con <code>class3</code>	74
B.3	Dendrograma de correlación entre características.	74
C.1	Visualización de los resultados obtenidos de las métricas en test por cada modelo.	75
C.2	Visualización de la matriz de confusión para el modelo RandomForest.	75

C.3	Visualización de la matriz de confusión para el modelo XGBoost.	76
C.4	Visualización de la matriz de confusión para el modelo ExtraTrees.	76
D.1	Alertas recibidas en Telegram.	77
D.2	Dashboard de monitorización en Grafana.	78
D.3	Logs del proceso de reentrenamiento.	79

List of Tables

2.1	Cantidad de instancias para cada ataque	15
2.2	Características básicas de tráfico de red.	18
2.3	Características generadas de tráfico de red.	19
2.4	Características de recursos del host.	20
2.5	Características de registros del host.	21
2.6	Variables objetivo del conjunto de datos.	22
2.7	Pares de características con correlación superior a 0,97.	27
3.1	Resumen de particiones utilizadas en el Hito 3.	38
3.2	Configuración principal de los modelos entrenados en el Hito 3.	38
3.3	Resultados globales en entrenamiento, validación y test.	41
3.4	Resultados finales en test.	45
3.5	Métricas operativas sobre el conjunto de prueba.	45
E.1	Recursos Hardware utilizados para el entrenamiento.	80

En los últimos años, la transformación digital del sector industrial ha dado lugar a entornos altamente interconectados donde la monitorización, automatización y análisis de datos desempeñan un papel central en la operación de los sistemas productivos [1]. La incorporación de tecnologías basadas en el *Industrial Internet of Things* (IIoT) ha permitido mejorar la eficiencia operativa, optimizar procesos y habilitar modelos de mantenimiento predictivo.

Sin embargo, esta creciente digitalización también introduce nuevos desafíos en materia de ciberseguridad. La convergencia entre redes IT (*Information Technology*) y OT (*Operational Technology*), junto con la exposición a infraestructuras basadas en IP, amplía significativamente la superficie de ataque de las organizaciones industriales.

En este contexto, la detección temprana de comportamientos anómalos en redes industriales se convierte en un elemento crítico para garantizar la continuidad operativa y la protección de infraestructuras críticas.

El presente trabajo aborda el diseño de un sistema de detección de anomalías basado en técnicas de Inteligencia Artificial, concebido como un servicio integrable dentro de un entorno IIoT. El proyecto se centra no solo en la construcción del modelo, sino en su definición como producto viable, evaluando su impacto técnico, operativo y organizativo.

Hito 1: Definición del problema y determinación del alcance

1.1 Definición del Problema

1.1.1 Contexto empresarial

En el marco de la denominada Industria 4.0, entendida como la cuarta revolución industrial caracterizada por la digitalización e interconexión inteligente de los sistemas productivos mediante tecnologías como el *Industrial Internet of Things* (IIoT) [2], la analítica de datos y la inteligencia artificial, los entornos industriales modernos incorporan sistemas basados en el IIoT, donde múltiples dispositivos, sensores y sistemas de control intercambian información de forma continua a través de redes industriales y corporativas. Esta interconexión permite optimizar procesos, mejorar la eficiencia operativa y facilitar la monitorización remota de activos críticos.

Sin embargo, el incremento de la conectividad y la exposición a redes IP amplía considerablemente la superficie de ataque. Los mecanismos tradicionales de seguridad, basados en reglas estáticas o firmas conocidas [3], presentan limitaciones a la hora de detectar ataques desconocidos o comportamientos anómalos que no se ajustan a patrones previamente definidos.

En este contexto, surge la necesidad de incorporar mecanismos inteligentes capaces de analizar grandes volúmenes de datos de red y operación industrial para identificar desviaciones respecto al comportamiento normal del sistema. La detección temprana de anomalías resulta fundamental para prevenir interrupciones del servicio, minimizar pérdidas económicas y proteger infraestructuras críticas [4].

Además, cualquier solución implementada debe considerar no solo la capacidad de detección, sino también el impacto operativo sobre el equipo humano encargado de la supervisión de la seguridad.

1.1.2 Formulación técnica del problema

Desde el punto de vista técnico, el problema se plantea como un sistema de detección de anomalías aplicado a datos generados en entornos IIoT (*Industrial Internet of Things*).

El conjunto de entrada X está compuesto por registros de actividad de red y variables asociadas al comportamiento de los dispositivos industriales, tales como logs de comunicación, tráfico industrial (MQTT, Modbus, OPC-UA), direcciones IP, puertos, frecuencia de paquetes y métricas de sensores.

El sistema debe generar como salida Y una clasificación de cada evento como *normal* o correspondiente a un tipo específico de ataque presente en el conjunto de datos. En este sentido, el problema se enmarca conceptualmente como una tarea de detección de anomalías, cuyo objetivo es distinguir entre comportamientos normales y anómalos en la red.

No obstante, esta detección se implementa mediante un enfoque de clasificación multiclase, en el que cada instancia no solo se clasifica como normal o anómala, sino que, en caso de ser anómala, se identifica el tipo concreto de ataque. De este modo, la detección de anomalías queda implícita en el proceso de clasificación: las instancias clasificadas como *normal* representan comportamiento legítimo, mientras que aquellas asignadas a cualquier clase de ataque son consideradas anomalías.

Este enfoque resulta especialmente adecuado cuando se dispone de datos etiquetados, ya que permite combinar la capacidad de detección con una caracterización detallada de las amenazas, facilitando su análisis y posible mitigación.

No obstante, en muchos entornos industriales reales no se dispone de un volumen amplio de datos etiquetados de ataques, por lo que el problema también puede abordarse mediante enfoques de aprendizaje no supervisado o híbrido, en los que el modelo aprende patrones de normalidad y detecta desviaciones significativas respecto a ellos.

El objetivo principal del sistema es maximizar la capacidad de detección de anomalías reales y, simultáneamente, minimizar la tasa de falsas alarmas, garantizando un equilibrio adecuado entre sensibilidad (*recall*) y precisión (*precision*).

1.1.3 Criterios de éxito

El sistema será considerado exitoso si cumple los siguientes criterios:

- Presenta una alta tasa de detección de anomalías reales, reduciendo al mínimo la probabilidad de que un ataque pase desapercibido, medida a través de métricas como el *recall* o tasa de verdaderos positivos sobre el conjunto de datos de evaluación.
- Mantiene una tasa de falsos positivos baja, evitando la generación excesiva de alertas innecesarias, evaluada mediante la *False Positive Rate* y la métrica de *precision*.
- Reduce la carga operativa del equipo de seguridad, evitando que los operadores deban supervisar continuamente incidencias irrelevantes, lo cual se estimará indirectamente mediante la disminución del número total de alertas generadas y de la proporción de eventos clasificados erróneamente como anómalos.
- Contribuye a disminuir el coste humano y económico asociado a la gestión manual de alarmas, derivado de la reducción de falsas alarmas y del tiempo estimado de revisión por parte de los operadores.

El sistema será considerado insuficiente si genera un volumen elevado de falsas alarmas que obligue a una supervisión constante por parte de los operadores, reflejado en una tasa alta de falsos positivos, o si no logra detectar comportamientos anómalos relevantes que puedan comprometer la operación industrial, evidenciado por un *recall* insuficiente en el entorno experimental evaluado.

1.1.4 Delimitación del problema

El sistema propuesto se concibe como una herramienta de apoyo a la detección de anomalías en entornos IIoT, centrada exclusivamente en el análisis de datos de red y comportamiento digital de los dispositivos industriales. No pretende sustituir completamente al equipo de seguridad ni eliminar la supervisión humana, sino asistir en la identificación temprana de posibles incidentes.

Asimismo, el sistema no contempla la ejecución automática de acciones correctivas irreversibles sobre la infraestructura industrial, tales como el apagado de dispositivos o la modificación directa de configuraciones críticas que puedan comprometer la continuidad operativa. Su función principal se limita a la generación de alertas fundamentadas y contextualizadas, las cuales deben ser evaluadas y validadas por el personal especializado antes de adoptar cualquier medida correctiva.

Quedan fuera del alcance del presente trabajo los aspectos relacionados con la seguridad física de las instalaciones, la protección perimetral tradicional o el análisis forense posterior al incidente. El enfoque se limita a la detección temprana basada en análisis de datos mediante técnicas de Inteligencia Artificial.

1.1.5 Limitaciones del estudio

El sistema desarrollado será evaluado utilizando el conjunto de datos X-IIoTID, el cual, aunque representa tráfico realista de entornos industriales, no sustituye completamente la validación en un entorno productivo real. En consecuencia, las métricas de rendimiento obtenidas reflejan el comportamiento del modelo bajo condiciones experimentales controladas.

Si bien el conjunto de datos empleado dispone de etiquetas que permiten realizar una evaluación cuantitativa objetiva, en un entorno industrial real dichas etiquetas no suelen estar disponibles de forma inmediata. En escenarios productivos, la validación del sistema requeriría mecanismos adicionales como la supervisión por parte de operadores especializados, procesos de etiquetado manual o sistemas de retroalimentación continua que permitan contrastar las alertas generadas.

Por tanto, los resultados obtenidos en este trabajo deben interpretarse como una validación experimental del enfoque propuesto, constituyendo un paso previo necesario antes de su eventual aplicación en un entorno operativo real.

1.2 Objetivos del Proyecto

1.2.1 Objetivo general

La creciente exposición de las infraestructuras industriales a ciberataques sofisticados, combinada con la heterogeneidad de dispositivos y protocolos que caracteriza a los entornos IIoT, hace necesario el desarrollo de soluciones inteligentes capaces de detectar amenazas de forma automática y comprensible. En este contexto, el objetivo general del proyecto es desarrollar y evaluar un sistema de detección de intrusiones sobre el conjunto de datos X-IIoTID [5], el cual dispone de etiquetas que permiten realizar una evaluación cuantitativa en un entorno experimental controlado. Para ello, se combinarán técnicas de aprendizaje automático supervisado, no supervisado y aprendizaje profundo, con el fin de identificar y clasificar ataques cibernéticos en entornos IIoT de manera precisa y explicable, considerando que en escenarios industriales reales la disponibilidad de datos etiquetados puede ser limitada.

1.2.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se articula en torno a los siguientes objetivos específicos:

1. **Adquirir un conocimiento profundo del dominio IIoT**, incluyendo sus protocolos de comunicación más habituales (MQTT, CoAP, WebSocket), la taxonomía de ataques presentes en este tipo de redes y los desafíos particulares que plantea la detección de intrusiones en infraestructuras críticas.

2. **Explorar el conjunto de datos X-IIoTID** mediante un análisis exploratorio exhaustivo, estudiando la distribución de clases, la naturaleza de las características de tráfico de red y registros de host, y detectando posibles inconsistencias que deban corregirse antes del modelado.
3. **Preparar los datos de forma rigurosa**, aplicando técnicas de limpieza, normalización, reducción de dimensionalidad y selección de características relevantes, con el objetivo de maximizar la calidad de la información que reciben los modelos durante el entrenamiento.
4. **Implementar y comparar múltiples enfoques de modelado**, incluyendo algoritmos de aprendizaje supervisado, técnicas no supervisadas y redes neuronales, abordando el problema como una tarea de clasificación multicategoría orientada a identificar el tipo específico de ataque presente en cada evento.
5. **Evaluar el rendimiento de los modelos** con métricas adecuadas al contexto de seguridad (*accuracy*, *precision*, *recall*, *F1-score* y *AUC-ROC*), prestando especial atención al equilibrio entre la tasa de detección y la tasa de falsos positivos, e incorporando métricas de impacto operativo derivadas de la matriz de confusión, como la reducción del volumen de alertas que requieren revisión manual ($Alertas_{manuales} = TP + FP$), donde TP y FP corresponden a los verdaderos positivos y falsos positivos respectivamente, y la estimación del coste asociado a la gestión de falsas alarmas ($Coste_{FP} = FP \cdot t_{rev} \cdot C_{hora}$), donde t_{rev} representa el tiempo medio de revisión de una alerta por parte de un operador y C_{hora} el coste por hora del personal encargado de dicha revisión. Adicionalmente, se considera como métrica de negocio el coste total operativo asociado a la gestión de alertas ($Coste_{total} = (TP + FP) \cdot t_{rev} \cdot C_{hora}$), lo que permite estimar el impacto económico global del sistema en un entorno real.
6. **Aplicar técnicas de explicabilidad** sobre los modelos desarrollados, identificando las características más influyentes en la detección de amenazas y facilitando la interpretación de los resultados por parte de los equipos de seguridad.
7. **Concebir el sistema con visión de producto**, diseñando una arquitectura que contemple no solo el entrenamiento de modelos sino también su despliegue, integración en un entorno operativo y monitorización a lo largo del tiempo.

1.3 Determinación del Alcance

1.3.1 Planificación (Cronograma)

El proyecto se desarrollará siguiendo las fases del ciclo de vida de un sistema ML, estructurándose, en cinco hitos bien marcados:

- **Hito 1: Definición del problema y alcance.** Donde se desarrollará las bases del proyecto y esquemas a utilizar y seguir.
- **Hito 2: Análisis y procesamiento de datos.** Donde se realizará una investigación del conjunto de datos a utilizar además de un preprocesamiento del mismo para que los modelos desarrollados en hitos siguientes puedan procesarlos. (Limpieza, transformación... entre otros)
- **Hito 3: Modelado y evaluación.** Donde realizaremos distintos tipos de estructuras y algoritmos para enfrentar el problema para más tarde, evaluar cada uno de ellos con las métricas definidas para el problema para obtener las mejores configuraciones.
- **Hito 4: Despliegue.** Donde realizaremos el despliegue e integración del sistema una vez obtenido el modelado de la solución del problema al que nos estamos enfrentando.
- **Hito 5: Monitorización e integración.** Donde tras realizar la integración, realizaremos varios análisis y una monitorización exhaustiva para observar su funcionamiento en producción. Tratando posibles problemas que puedan invocarse durante esta fase.

1.3.2 Viabilidad económica

Para este proyecto, el coste estimado del sistema incluye:

- Infraestructura cloud para entrenamiento.
- Recursos computacionales.
- Tiempo de desarrollo del equipo. (4 personas)
- Obtención y preprocesamiento de los datos. (En nuestro caso, el conjunto de datos X-IIoTID es gratuito debido a su disponibilidad en la plataforma **Kaggle**. Pero, en el caso de requerir un aumento, habría que asumir un mayor coste)
- Desarrollo de diferentes estructuras de modelado y algoritmos.
- Análisis exhaustivo de los resultados obtenidos.
- Desarrollo de 3 meses y medio de duración.

Teniendo todo lo anterior en cuenta, tratándose de un desarrollo a baja/media escala, la realización de todo el proceso completo de preprocesamiento, entrenamiento en cloud, utilización de recursos y creación del servicio se estima en un coste aproximado de 25000 € [6][7]. Este coste incluye principalmente el uso de infraestructura cloud (instancias de cómputo para entrenamiento y almacenamiento de datos), consumo de recursos computacionales (CPU/GPU), así como los costes asociados al desarrollo técnico y despliegue

del sistema. Por otra parte, considerando el trabajo del equipo, compuesto por cuatro miembros dedicando aproximadamente 10 horas semanales durante 16 semanas, con un coste medio de 40 euros por hora [8], se obtiene el siguiente coste asociado a recursos humanos:

- **Coste del servicio completo (infraestructura, recursos y desarrollo técnico):** 25.000 € [6][7].
- **Coste del equipo de trabajo:** 25.600 €, calculado a partir de 4 miembros dedicando 10 horas semanales durante 16 semanas, con un coste medio de 40 euros por hora [8].
- **Coste total estimado del proyecto:** 50.600 €.

El retorno de la inversión (ROI) puede estimarse mediante:

$$ROI = \frac{\text{Beneficio esperado} - \text{Coste del sistema}}{\text{Coste del sistema}}$$

Dado que un incidente de seguridad industrial puede suponer pérdidas muy variables (llegando a alcanzar dependiendo de la gravedad una oscilación entre 5-10 millones de euros) [9], la prevención de un único ataque grave en un entorno complejo como en la industria nuclear, justificaría la inversión en el sistema de detección. No obstante, haremos el cálculo para comprobar de manera precisa el ROI resultante...

$$ROI = \frac{5000000 - 25600}{25600} = 194,3125 * 100 = 19431,25\%$$

Como vemos, para este ejemplo, en el proyecto tendríamos un ROI muy elevado de 19431,25%. Teniendo en cuenta lo costoso que sería no detectar una intrusión de tal calibre en este campo. No obstante, haremos el cálculo del ROI en el caso de aplicar la solución en un ámbito no tan crítico como en la fabricación de productos electrónicos donde una detección tardía puede abordar una pérdida de unos 86000 € [9].

$$ROI = \frac{86000 - 50600}{50600} = 0,6996 * 100 = 69,96\%$$

En este caso, obtendríamos un ROI positivo del 69,96%. Tras los cálculos de los dos ejemplos, se puede concluir que, el sistema propuesto presenta un ROI positivo tanto para empresas cuya operación requiere una detección de anomalías de carácter crítico, como para aquellas en las que dicha necesidad es menos urgente.

1.3.3 Valor

El valor del sistema puede estructurarse en diferentes secciones ya que, con su salida, es capaz de otorgar no solo un gran valor a nivel tecnológico, sino también a nivel económico para las empresas que hagan uso de el y, entre otros, organizacional y reputacional.

1.3.4 Valor tecnológico

Desde la perspectiva tecnológica, el sistema a desarrollar aporta gran valor al utilizar estructuras y algoritmos de aprendizaje automático y profundo. A diferencia de los tradicionales utilizados en este ámbito de detección de intrusiones que hacen uso reglas estáticas o firmas conocidas. Capaz además, de poder adaptarse no solo a una posible estructura empresarial si no varias que hagan uso de los sistemas basados en IIoT.

1.3.5 Valor económico

Como hemos comentado anteriormente, este sistema puede ayudar a las empresas que lo integren a obtener detecciones precisas de anomalías o intrusiones que provoquen fallas de seguridad en sus tecnologías. Resguardándolas, de pérdidas provocadas por estas fallas de hasta más de 9 millones de euros (según la gravedad de la falla sin tratar).

1.3.6 Valor organizacional

Al permitir un control preciso y monitorización en este tipo de fallas de seguridad, las empresas que integren el sistema podrían disponer de tecnologías internas más fáciles de mantener para sus trabajadores, reduciendo así pérdidas significativas de tiempo y carga operativa.

1.3.7 Valor reputacional

Es importante tener en cuenta que las tecnologías IIoT, cuando se ven afectadas por una intrusión o anomalía, no solo pueden generar pérdidas económicas para las empresas, sino que también pueden suponer un riesgo para los trabajadores que estén monitorizando esas tecnologías. Por ello, la implementación del sistema, no solo contribuiría a minimizar el impacto económico, permitiría establecer una infraestructura tecnológica interna más segura para todos sus integrantes.

1.3.8 Plan de actuación ante detección

En caso de detectar una anomalía:

- Generación automática de alerta.
- Registro del evento en el sistema.
- Aislamiento del nodo comprometido (si procede).
- Notificación al equipo de seguridad.

1.3.9 Roles del equipo

Para terminar, el equipo estará compuesto por Iván Román, Rafael Sánchez, Marcelo Chinarro y Raúl Martín los cuáles, cubrirán los diferentes roles del proyecto:

- **Expertos en la materia y en el dominio del problema.** Responsables de definir las necesidades del negocio, formular las preguntas clave y establecer los objetivos estratégicos del proyecto. El miembro del equipo asignado a este rol es **Rafael Sánchez**.
- **Científicos de datos.** Encargados del diseño, desarrollo y validación de los modelos de aprendizaje automático que permitan dar respuesta a los requerimientos definidos. El miembro del equipo asignado a este rol es **Marcelo Chinarro**.
- **Ingenieros de datos.** Responsables de la adquisición, transformación, almacenamiento y preparación de los datos necesarios, garantizando su calidad y disponibilidad para el modelado. El miembro del equipo asignado a este rol es **Marcelo Chinarro**.
- **Ingenieros informáticos.** Encargados de la integración de los modelos desarrollados dentro de los sistemas, aplicaciones o infraestructuras empresariales de destino. El miembro del equipo asignado a este rol es **Iván Román y Rafael Sánchez**.
- **DevOps.** Responsables de la automatización, despliegue, mantenimiento y operación estable de las soluciones desarrolladas, asegurando su correcta integración continua y entrega continua (CI/CD). El miembro del equipo asignado a este rol es **Raúl Martín**.
- **Gestores y auditores.** Encargados de supervisar el cumplimiento de los requisitos normativos, legales, éticos y de calidad establecidos para el proyecto. El miembro del equipo asignado a este rol es **Iván Román**.
- **Arquitectos de Machine Learning.** Responsables del diseño de la arquitectura global de las soluciones de aprendizaje automático, garantizando su escalabilidad, robustez y correcta monitorización desde la fase de diseño hasta su operación en producción. El miembro del equipo asignado a este rol es **Raúl Martín**.

1.4 Conclusiones

En este primer hito se ha conseguido aterrizar el problema que supone la creciente exposición de los entornos IIoT a riesgos de ciberseguridad, especialmente en un contexto donde los sistemas tradicionales ya no son suficientes para detectar amenazas nuevas o comportamientos fuera de lo normal.

A lo largo del trabajo se han definido los objetivos del proyecto y su alcance, dejando claro que el sistema está pensado como un apoyo para los equipos de seguridad, no como un reemplazo. También se ha revisado su viabilidad, viendo que la inversión necesaria es pequeña si se compara con el impacto que puede tener prevenir un incidente grave.

En definitiva, este hito permite establecer una base sólida para avanzar en las siguientes fases, centradas ya en el análisis de datos y el desarrollo de modelos. Con ello, se marca una dirección clara para construir una solución que realmente aporte valor en la detección temprana de intrusiones en entornos industriales.

Hito 2: Los datos: origen, descripción y análisis

2.1 Introducción

En cualquier proyecto relacionado con inteligencia artificial y aprendizaje automático, el análisis de los datos con los que trabajar se trata de un punto vital. Debido a que es de suma importancia entender y conocer con gran certeza los datos con los que estamos trabajando para saber exactamente que técnicas aplicar y cuáles no. Adicionalmente, es importante considerar si el conjunto de datos disponible proporciona información suficiente para desarrollar la solución del problema o si, por el contrario, nos haría falta realizar algún tipo de aumento para alcanzar la solución deseada.

El conjunto de datos a analizar y utilizar para este proyecto se trata del llamado X-IIoTID (Intrusion Dataset for Iot and IIoT). Este dataset está formado con más de 800000 instancias y un total de 18 tipos de amenazas diferentes que detectar (siendo estas variables objetivo en las que nos tenemos que centrar). En este apartado, se desarrollarán y analizarán en profundidad todas las características que lo componen, desde su origen y sus características básicas hasta los detalles relacionados con sus datos.

Para ello, realizaremos un Análisis Exploratorio de los datos (EDA) que nos ayudará a entender y comprender todo el conjunto de datos y, cómo proceder en las siguientes fases del proyecto. Además, una vez realizado este análisis, seleccionaremos las mejores métricas de rendimiento a utilizar para el desarrollo correcto de los próximos modelos, justificando la elección de cada una según lo descubierto anteriormente.

2.2 Fuentes, origen y tamaño de los datos

2.2.1 Origen del conjunto de datos

El conjunto de datos X-IIoTID debe su origen en el año 2022 a la necesidad de conseguir sistemas de detección de anomalías en IIoT mucho más resistentes y robustos capaces de manejarse en sistemas heterogéneos con gran conectividad e interoperabilidad (acción muy complicada debido a la gran cantidad de sistemas que pueden componerse dentro de las empresas tecnológicas al igual que la cantidad de protocolos manejados para ello).[10]

Por ello, gracias a la creación de este conjunto de datos, es posible desarrollar modelos de detección de anomalías independientes de la conectividad y del tipo de dispositivo, ya que está compuesto por numerosos casos posibles y actuales de ataques o fallos en sistemas IIoT.

2.2.2 Descripción del conjunto de datos

El conjunto a utilizar está compuesto de numerosos casos relacionados con fallas durante procedimientos y ataques en sistemas IIoT. En específico, posee una cantidad de 9 categorías de ataques entre los que encontramos: extracción de datos (*Exfiltration*), manipulación de datos (*Tampering*), ataques de extorsión (*RDoS*), recopilación de datos (*Reconnaissance*), cifrado de archivos (*Crypto ransomware*), canales de comunicación para el atacante (*Command and control*), expansión dentro de la red (*Lateral movement*), desarrollo de *exploits* (*Weaponization*) y el uso de vulnerabilidades del sistema (*Exploitation*).

Y, de forma más específica, tenemos un total de 18 tipos de ataques entre todas las categorías, siendo estos los que queremos identificar en nuestra solución, por lo que, al incluir también las instancias sin ningún tipo de ataque, la variable objetivo estará compuesta por un total de 19 clases.

Cada una de las instancias del conjunto de datos está formada por un total de 65 características, teniendo:

- **Características del tráfico de red básicas (16).** Como la fecha de registro (*Date*), la marca de tiempo (*Timestamp*) o el protocolo (*Protocol*).
- **Características generadas del tráfico (16).** Como el estado de la conexión (*Conn_state*), los bytes totales por segundo (*byte_rate*) o el número total de paquetes por segundo (*packet_rate*).
- **Características de recursos de host (24).** Como el tiempo promedio del usuario (*Avg_user_time*), el tiempo promedio del sistema (*Avg_system_time*) o el tiempo promedio de solicitudes de transferencia (*Avg_tps*).
- **Características de registros de host (9).** Como la alerta de anomalía (*anomaly_alert*), la actividad de archivo (*File_activity*) o los privilegios de la actividad realizada (*is_privileged*).

2.2.3 Formato y tipos de los datos

El formato de representación del conjunto de datos se trata de uno tabular con extensión CSV. Por otra parte, centrándonos en las características, una vez observadas todas, podemos apreciar un total de 25 variables discretas (donde tenemos 15 de ellas en formato binario) y 40 variables continuas (número de paquetes, número de bytes, porcentajes, desviaciones estándar y promedios de memoria, tiempos, tareas y cambios).

2.2.4 Tamaño del conjunto de datos

El tamaño que tenemos en el conjunto de datos es de un total de 820824 instancias. Pero, debido a la solución que queremos abarcar, tenemos que saber de forma precisa las instancias que componen cada uno de los ataques dentro de sus categorías correspondientes. Con ello, sabremos con certeza la distribución de clases en el conjunto de datos y, cómo actuar a la hora de la realización del modelado. En la **Tabla 2.1** puede observarse la distribución.

Categoría	Tipo	Instancias	Total	% del total
Exfiltration	Exfiltration	22134	22134	2.7%
Tampering	False data injection	5094	5122	0.62%
	Fake notification	28		
Reconnaissance	Generic scanning	50277	127590	15.54%
	Scanning vulnerabilities	52852		
	Fuzzing	23148		
	Discovering resources	1313		
Crypto ransomware	Crypto ransomware	458	458	0.06%
RDoS	RDoS	141261	141261	17.21%
Command and control	Command and control	2863	2863	0.35%
Lateral movement	MQTT cloud broker-subscription	23524	31596	3.85%
	Modbus-register reading	5953		
	TCP relay	2119		
Exploitation	Reverse shell	1016	1133	0.14%
	Man-in-the-middle	117		
Weaponization	Brute-force	47241	67260	8.19%
	Dictionary	2572		
	Insider malicious	17447		

Table 2.1: Cantidad de instancias para cada ataque

Como se observa en la tabla, el porcentaje de ataques del dataset es de un 48.66% mientras que todo el resto de instancias, específicamente un 51.34%, se tratan de solicitudes normales. En la **Figura 2.1** puede observarse la distribución de forma visual.

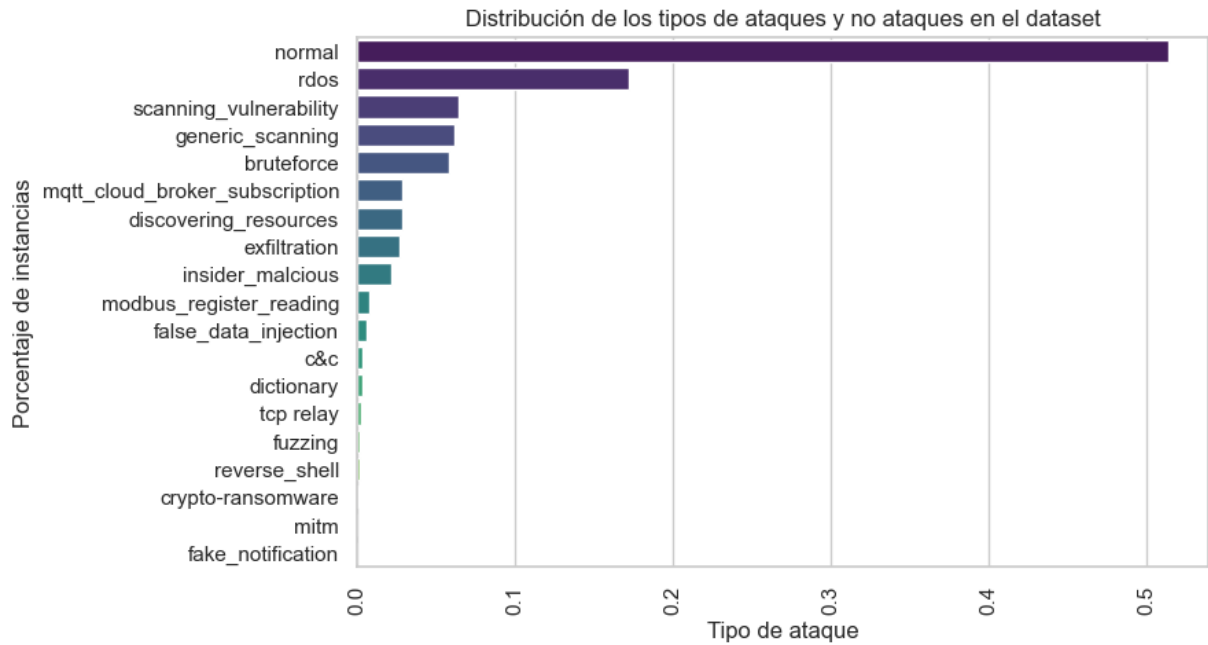


Figure 2.1: Distribución de los porcentajes de ataques en el dataset X-IIoTID.

2.2.5 Naturaleza del problema

Al querer predecir el tipo concreto de anomalía en cada instancia, estamos ante un problema de clasificación multiclase. Y, como se puede apreciar en la tabla de la anterior sección, las variables objetivo presentan un gran desbalanceo en su representación en el conjunto de datos. Debido a ello, a la hora de evaluar el rendimiento de los modelos, haremos uso del promediado *macro* (*macro averaging*), el cual consiste en calcular la métrica de forma independiente para cada clase y posteriormente promediar los resultados, otorgando el mismo peso a todas las clases por igual. Todo ello permite evitar sesgos hacia las clases mayoritarias y obtener una evaluación más representativa del rendimiento del modelo en presencia de desbalanceo entre clases.

2.3 Inventario de datos

En esta sección se describe en detalle el conjunto de variables que componen el dataset X-IIoTID, organizándolas según su origen y naturaleza. Dicho inventario constituye la base sobre la que se apoyará el análisis exploratorio posterior y orientará las decisiones de preprocesamiento y selección de características en las fases siguientes del proyecto.

2.3.1 Descripción de las variables

El conjunto de datos X-IIoTID está compuesto por un total de 68 variables, de las cuales 65 son características predictoras y 3 corresponden a las variables objetivo. Las características predictoras se agrupan en cuatro categorías según su procedencia:

- **Características básicas de tráfico de red (16):** capturan información sobre las comunicaciones a nivel de conexión, incluyendo direcciones IP de origen y destino, puertos, protocolo, duración de la conexión, volumen de bytes y paquetes intercambiados, y métricas sobre la transmisión.
- **Características generadas de tráfico de red (16):** derivadas de las anteriores, proporcionan información sobre el estado de la conexión, tasas de transmisión, proporciones de tráfico y banderas de los paquetes (SYN, ACK, FIN, RST, etc.).
- **Características de recursos del host (24):** recogen métricas sobre el rendimiento del sistema operativo en ventanas de 10 segundos, incluyendo tiempos de ejecución de procesos, tiempos de espera de E/S, carga del sistema, uso de memoria y cambios de contexto.
- **Características de registros del host (9):** incluyen alertas de seguridad generadas por herramientas de monitorización como Zeek y OSSEC, junto con información sobre actividad de archivos, procesos, intentos de inicio de sesión y accesos privilegiados.

En cuanto al tipo de dato, el conjunto presenta **25 variables discretas**, de las cuales 15 son de naturaleza binaria (valores 0 o 1), y **40 variables continuas** que recogen métricas como número de paquetes, bytes, porcentajes, desviaciones estándar y promedios de tiempos y carga del sistema.

Las tres variables objetivo estructuran la clasificación en distintos niveles jerárquicos: la primera identifica si el tráfico es normal o anómalo (clasificación binaria), la segunda clasifica la categoría del ataque (9 categorías) y la tercera especifica el tipo exacto de ataque (18 tipos).

2.3.2 Diccionario de variables

A continuación se presentan las tablas descriptivas de cada grupo de variables, indicando el nombre, tipo y descripción de cada característica.

Nombre	Tipo	Descripción
Date	Discreta	Fecha de registro. (Ordinal)
Timestamp	Discreta	Marca de tiempo. (Ordinal)
Scr_IP	Discreta	Dirección IP del origen.
Des_IP	Discreta	Dirección IP del destino.
Scr_port	Discreta	Puerto TCP/UDP del origen o código ICMP.
Des_port	Discreta	Puerto TCP/UDP del destino o código ICMP.
Protocol	Discreta	Protocolo. (TCP, UDP, ICMP u otro) (Categórica)
Service	Discreta	Protocolo de aplicación en el puerto de destino. (Categórica)
Duration	Continua	Diferencia de tiempo entre el primer y último paquete visto. Int: [0, 990]
Scr_bytes	Continua	Número de bytes enviados desde el origen al destino. Int: [0, 1665228]
Des_bytes	Continua	Número de bytes enviados desde el destino al origen. Int: [0, 19359682]
missed_bytes	Continua	Número de bytes perdidos en interrupciones de contenido. Int: [0, 1882937]
Scr_pkts	Continua	Número de paquetes enviados. Int: [0, 65541]
Des_pkts	Continua	Número de paquetes recibidos. Int: [0, 65575]
Scr_ip_bytes	Continua	Bytes enviados en el campo de longitud total del encabezado IP. Int: [0, 9999]
Des_ip_bytes	Continua	Bytes recibidos en el campo de longitud total del encabezado IP. Int: [0, 20110090]

Table 2.2: Características básicas de tráfico de red.

Nombre	Tipo	Descripción
Conn_state	Discreta	Estado de conexión (1: completa, 2: en espera, 3: parcial) (Categórica)
total_bytes	Continua	Total de bytes intercambiados entre origen y destino. Int: [56, 39163466]
byte_rate	Continua	Total de bytes por segundo. Int: [1, 306000000]
total_packet	Continua	Número total de paquetes intercambiados. Int: [1, 131116]
paket_rate	Continua	Número total de paquetes por segundo. Int: [0, 3000000]
Scr_bytes_ratio	Continua	Porcentaje de bytes enviados respecto al total. Int: [0, 1]
Des_bytes_ratio	Continua	Porcentaje de bytes recibidos respecto al total. Int: [0, 1]
Scr_packts_ratio	Continua	Porcentaje de paquetes enviados respecto al total. Int: [0, 1]
Des_pkts_ratio	Continua	Porcentaje de paquetes recibidos respecto a los del origen. Int: [0, 1]
is_syn_only	Discreta	1 si la conexión tiene un paquete con bandera SYN (Binaria)
Is_SYN_ACK	Discreta	1 si la conexión tiene un paquete con bandera SYN-ACK (Binaria)
is_pure_ack	Discreta	1 si la conexión tiene un paquete con bandera ACK pura (Binaria)
is_with_payload	Discreta	1 si la conexión tiene un paquete con carga útil (Binaria)
FIN or RST	Discreta	1 si la conexión tiene un paquete con bandera FIN o RST (Binaria)
Bad_checksum	Discreta	1 si hay un paquete con suma de verificación incorrecta (Binaria)
is_SYN_with_RST	Discreta	1 si hay un paquete con ambas banderas SYN y RST (Binaria)

Table 2.3: Características generadas de tráfico de red.

Table 2.4: Características de recursos del host.

Nombre	Tipo	Descripción
Avg_user_time	Continua	Promedio del tiempo de usuario en los últimos 10 segundos. Int: [0.228, 0.228]
Std_user_time	Continua	Desviación estándar del tiempo de usuario. (ventana de 10s) Int: [0, 0.13]
Avg_nice_time	Continua	Promedio del tiempo de ajuste de prioridad en los últimos 10s. Int: [0, 21]
Std_nice_time	Continua	Desviación estándar del tiempo de nice. (ventana de 10s) Int: [0, 19]
Avg_system_time	Continua	Promedio del tiempo de sistema en los últimos 10 segundos. Int: [0, 74]
Std_system_time	Continua	Desviación estándar del tiempo de sistema. (ventana de 10s) Int: [-0.19, 32]
Avg_iowait_time	Continua	Promedio del tiempo de espera de E/S en los últimos 10s. Int: [0, 72]
Std_iowait_time	Continua	Desviación estándar del tiempo de espera de E/S. (ventana de 10s) Int: [-0.49, 56]
Avg_ideal_time	Continua	Promedio del tiempo inactivo en los últimos 10 segundos. Int: [0, 98]
Std_ideal_time	Continua	Desviación estándar del tiempo inactivo. (ventana de 10s) Int: [0, 60]
Avg_tps	Continua	Promedio de solicitudes de transferencia por segundo. (ventana 10s) Int: [-45, 1053]
Std_tps	Continua	Desviación estándar de solicitudes de transferencia/s. (ventana 10s) Int: [0, 533]
Avg_rtps	Continua	Promedio de transacciones de lectura por segundo. (ventana de 10s) Int: [0, 1047]
Std_rtps	Continua	Desviación estándar de transacciones de lectura/s. (ventana 10s) Int: [0, 531]
Avg_wtps	Continua	Promedio de transacciones de escritura por segundo. (ventana 10s) Int: [-46, 60]
Std_wtps	Continua	Desviación estándar de transacciones de escritura/s. (ventana 10s) Int: [-42, 144]

Continúa en la siguiente página...

Table 2.4 – Continuación

Nombre	Tipo	Descripción
Avg_ldavg_1	Continua	Promedio de la carga del sistema. (último minuto, ventana de 10s) Int: [0, 4]
Std_ldavg_1	Continua	Desviación estándar de la carga del sistema. (ventana de 10s) Int: [-0.163691, 1]
Avg_kbmemused	Continua	Promedio de memoria usada en kilobytes. (ventana de 10s) Int: [0, 936797]
Std_kbmemused	Continua	Desviación estándar de la memoria usada en KB. (ventana de 10s) Int: [0, 9995]
Avg_num_Proc/s	Continua	Promedio del número de tareas creadas por segundo. (ventana 10s) Int: [-11, 303432]
Std_num_proc/s	Continua	Desviación estándar de tareas creadas por segundo. (ventana 10s) Int: [-24, 429113]
Avg_num_cswch/s	Continua	Promedio de cambios de contexto por segundo. (ventana de 10s) Int: [-861, 21007]
std_num_cswch/s	Continua	Desviación estándar de cambios de contexto/s. (ventana de 10s) Int: [-1918, 7250]

Table 2.5: Características de registros del host.

Nombre	Tipo	Descripción
anomaly_alert	Discreta	1 si la conexión tiene una alerta de Zeek; 0 en caso contrario (Binaria)
OSSEC_alert	Discreta	1 si la conexión tiene una alerta de OSSEC; 0 en caso contrario (Binaria)
OSSEC_alert_level	Discreta	Nivel de gravedad de la alerta de OSSEC (Ordinal)
read_write_physical.process	Discreta	1 si hay actividad de lectura/escritura en el proceso físico; 0 si no (Binaria)
File_activity	Discreta	1 si hay actividad de archivo (leer, escribir, eliminar, copiar, crear, descargar); 0 si no (Binaria)
Process_activity	Discreta	1 si se ejecuta o inicia un nuevo proceso; 0 en caso contrario (Binaria)

Continúa en la siguiente página...

Table 2.5 – Continuación

Nombre	Tipo	Descripción
is_privileged	Discreta	1 si la actividad realizada es privilegiada; 0 en caso contrario (Binaria)
Login_attempt	Discreta	1 si hay un intento de inicio de sesión; 0 en caso contrario (Binaria)
Succesful_login	Discreta	1 si ocurre un inicio de sesión exitoso; 0 en caso contrario (Binaria)

Nombre	Tipo	Descripción
class1	Discreta	Clasificación binaria: 0 (tráfico normal), 1 (tráfico anómalo)
class2	Discreta	Clasificación por categoría de ataque (9 categorías: Reconnaissance, Weaponization, Exploitation, Lateral movement, Command and control, Exfiltration, Tampering, Crypto ransomware, RDoS)
class3	Discreta	Clasificación por tipo específico de ataque (18 tipos: Generic scanning, Brute-force, RDoS, etc.)

Table 2.6: Variables objetivo del conjunto de datos.

2.3.3 Selección de características relevantes

Dado que el objetivo del conjunto de datos X-IloTID es ser agnóstico al dispositivo y a la conectividad, una primera reducción de dimensionalidad consiste en eliminar las características directamente asociadas a la identificación de red y a la distribución de paquetes, ya que estas variables están ligadas a configuraciones específicas de red y hardware, lo que dificultaría la generalización del modelo a otros entornos IIoT. Las características eliminadas en este primer paso son: Scr_IP, Scr_port, Des_IP, Des_port, Scr_bytes, Des_bytes, Scr_pkts, Des_pkts, Scr_packets_ratio, Des_pkts_ratio, Scr_bytes_ratio, Des_bytes_ratio, Scr_ip_bytes, Des_ip_bytes, Timestamp y Date. Esta reducción inicial lleva el conjunto de 65 a 49 características.

Sobre las 49 características restantes se aplicarán tres criterios de eliminación adicionales durante la fase de preprocesamiento:

- **Eliminación por varianza nula:** las características con varianza igual a 0 no presentan variabilidad en los datos y no contribuyen a la diferenciación entre clases, por lo que serán descartadas (véase Figura B.1). En concreto, se eliminan las variables is_SYN_with_RST y Bad_checksum.

- **Eliminación por alta correlación entre características:** cuando dos características presentan una correlación superior a 0.97, se considera que contienen información redundante. Conservar ambas podría generar multicolinealidad, afectando la estabilidad del modelo. Se elimina una característica por cada par redundante identificado (véase Figura B.3). En este caso, se descartan las variables `is_pure_ack`, `FIN or RST`, `byte_rate`, `OSSEC_alert_level`, `Succesful_login`, `Process_activity` e `is_privileged`.
- **Eliminación por baja correlación con la variable objetivo:** las características con una correlación inferior a 0,025 respecto a la variable objetivo `class3` se consideran irrelevantes para la predicción y son descartadas (véase Figura B.2). En particular, se eliminan las variables `Std_rtps`, `Std_ldavg_1`, `Std_ideal_time`, `Std_kbmemused`, `Std_tps`, `Std_iowait_time`, `missed_bytes`, `Avg_wtps`, `Std_num_proc/s` y `Avg_num_Proc/s`.

Aplicando estos tres criterios de forma secuencial, el número de características se reduce de las 65 iniciales a **30 variables finales**, simplificando el modelo y manteniendo únicamente aquellas con mayor capacidad predictiva.

2.4 Análisis de los datos

2.4.1 Análisis exploratorio del dataset

El análisis exploratorio de los datos se ha llevado a cabo siguiendo la metodología CRISP-DM, concretamente su segunda fase de comprensión de los datos. El conjunto de datos X-IIoTID está compuesto por **820.834 instancias** en total, de las cuales 596.017 contienen información completa en todas sus características, mientras que 224.817 presentan valores nulos o inconsistentes que requerirán tratamiento en la fase de preprocesamiento.

Respecto a la recogida de datos, el tráfico sin anomalías fue capturado entre el 5 de diciembre de 2019 y el 23 de marzo de 2020 de forma no continua, mientras que los experimentos con tráfico anómalo se llevaron a cabo entre el 7 de enero y el 27 de marzo de 2020, repitiéndose cada tipo de ataque varias veces. Es importante destacar que las instancias normales y las de ataque no tienen correlación temporal entre sí, por lo que la detección debe basarse exclusivamente en el análisis de las características de cada instancia de forma independiente.

Con el objetivo de reducir la dimensionalidad del espacio de características de forma no supervisada, se ha aplicado un método de selección basado en PCA denominado *PCA Feature Subset Selection* (PCAFSS). En primer lugar, se ajusta un modelo PCA sobre el conjunto de entrenamiento reteniendo el **95% de la varianza explicada**, lo que da lugar a un total de n_{comp} **componentes principales**. Estas componentes representan

combinaciones lineales de las variables originales que capturan la mayor parte de la información presente en los datos.

A partir de estas componentes, se analizan sus vectores de carga (*loadings*), que cuantifican la contribución de cada variable original a cada componente principal. Para cada componente, se seleccionan las **20 variables con mayor contribución en valor absoluto**, es decir, aquellas que más influyen en la formación de dicha componente. De este modo, se identifican las variables más relevantes desde la perspectiva de cada dirección principal de variabilidad en los datos.

Posteriormente, se contabiliza la frecuencia con la que cada variable aparece entre las más relevantes a lo largo de todas las componentes. Las variables que aparecen con mayor frecuencia se consideran más representativas del conjunto de datos, ya que contribuyen de forma consistente a múltiples componentes principales.

Finalmente, las variables se ordenan según dicha frecuencia de aparición y se selecciona el subconjunto más representativo, dando lugar a un espacio final de **34 variables**. Este procedimiento permite identificar aquellas características originales con mayor capacidad explicativa global, evitando trabajar directamente con las componentes transformadas. Este enfoque preserva la interpretabilidad del modelo al mantener variables originales en lugar de combinaciones lineales latentes, y garantiza que el modelo recibe únicamente las variables con mayor capacidad representativa.

Dado que el análisis de importancia mediante Random Forest y la visualización de distribuciones (Sección 2.4.3) muestran un solapamiento significativo entre clases, se confirma la ausencia de separabilidad lineal y la necesidad de emplear algoritmos capaces de capturar relaciones no lineales.

2.4.2 Distribución de las clases

El conjunto de datos dispone de tres variables objetivo que estructuran la clasificación en distintos niveles.

- **Clasificación binaria** (`class3`).

En el conjunto de entrenamiento (70%, 574.583 registros), la distribución de `class3` es prácticamente equilibrada: un **51,34%** de instancias normales y un **48,66%** de instancias anómalas, como se muestra en la Figura 2.2. Este balance favorece el entrenamiento sin necesidad de técnicas de ajuste como sobremuestreo o submuestreo.

- **Clasificación por categoría** (`class2`) y **por tipo** (`class1`).

La distribución de las 9 categorías y los 18 tipos concretos de ataque muestra un marcado desequilibrio. RDoS es la categoría más representada (35,39%), seguida de Reconnaissance (32,00%) y

Weaponization (16,83%). En el extremo opuesto, Exploitation (0,28%) y Crypto ransomware (0,11%) tienen una presencia mínima. La Tabla 2.1 recoge la distribución completa desglosada por tipo de ataque.

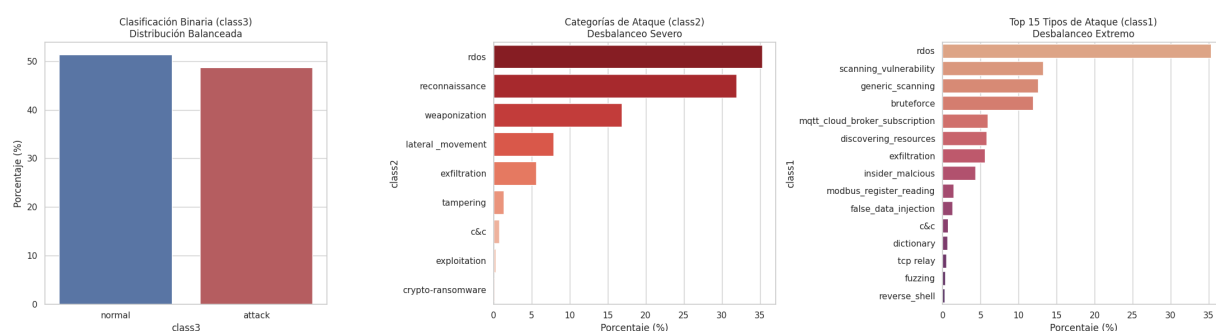


Figure 2.2: Distribución del tráfico en el dataset X-IIoTID. A la izquierda, la proporción casi igualitaria entre tráfico normal (51,34%) y anómalo (48,66%) en la clasificación binaria (class3) constituye un escenario favorable para el entrenamiento de clasificadores. A la derecha, las distribuciones por categoría (class2) y tipo de ataque (class1) muestran un marcado desequilibrio que deberá abordarse en la fase de modelado.

En el desglose por tipo específico (class1), el desequilibrio se acentúa aún más: tipos como Man-in-the-middle (0,03%) y Fake notification (0,01%) tienen una representación prácticamente testimonial. Dado este desequilibrio, se utilizará el promediado **MACRO** durante la evaluación, otorgando el mismo peso a todas las clases independientemente de su frecuencia.

2.4.3 Análisis estadístico de las variables

El análisis de varianza permite identificar las características con valores casi constantes que aportan poca información discriminativa. Entre las 15 con menor varianza se encuentran principalmente características binarias y discretas: is_SYN_with_RST, Bad_checksum, Std_ldavg_1, OSSEC_alert, File_activity, is_privileged, Process_activity, Succesful_login, Login_attempt, anomaly_alert, is_with_payload, Conn_state, read_write_physical.process, Is_SYN_ACK e is_pure_ack.

El análisis de distribuciones revela una marcada asimetría positiva con colas largas, característica habitual del tráfico de red. La Figura 2.3 muestra la distribución de paket_rate en escala logarítmica. Se observa que ambas clases presentan un solapamiento significativo a lo largo de todo el rango de valores, lo que confirma que ninguna variable individual es suficiente para discriminar el tráfico malicioso y que se requiere un enfoque multivariante basado en aprendizaje automático.

Finalmente, el análisis de importancia mediante Random Forest revela que Duration, total_bytes, read_write_physical.process y Avg_num_cswch/s son las variables más influyentes. Destaca que Duration

presenta una correlación lineal muy baja con la variable objetivo, pero resulta ser la característica más importante según Random Forest, ilustrando su capacidad para capturar relaciones no lineales que el análisis clásico de correlación no detecta.

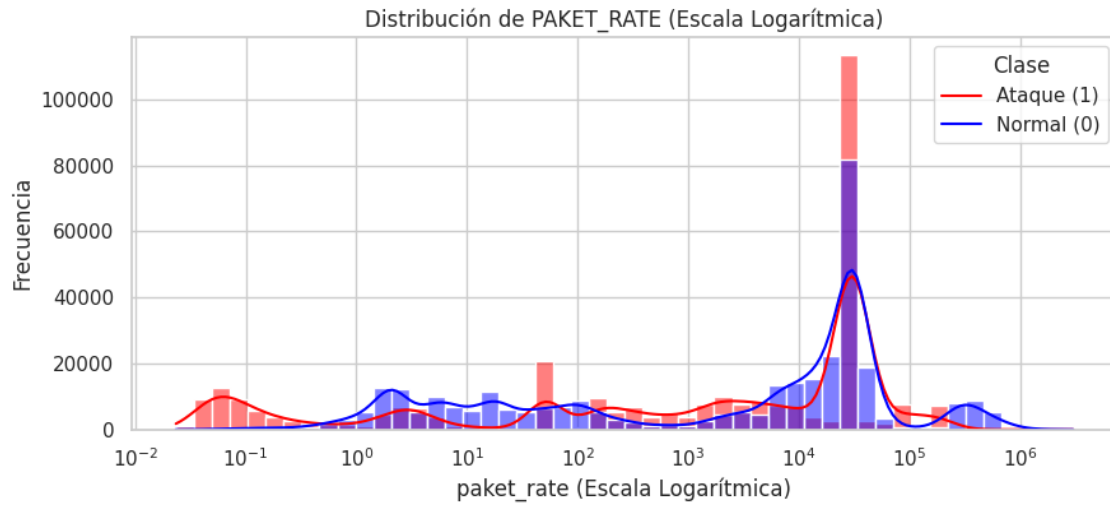


Figure 2.3: Distribución de `paket_rate` en escala logarítmica para tráfico normal (azul) y anómalo (rojo). El solapamiento entre distribuciones evidencia la insuficiencia de reglas estáticas basadas en umbrales individuales.

2.4.4 Correlación y selección de características

La selección de características se ha abordado en dos etapas: un triple filtro estadístico aplicado sobre el conjunto de entrenamiento, seguido del PCAFSS descrito en la Sección 2.4.1.

Eliminación de variables agnósticas. Como decisión de diseño previa a cualquier filtro estadístico, se descartan las 16 características ligadas a la topología específica de la red de pruebas: direcciones IP y puertos de origen y destino, ratios de bytes y paquetes, y marcas temporales. Su inclusión comprometería la generalización del modelo a otros entornos IIoT, al memorizarse la infraestructura concreta en lugar de aprenderse patrones de comportamiento.

Triple filtro estadístico. Sobre las características restantes se aplican secuencialmente tres criterios de eliminación:

- **Filtro de varianza:** Se eliminan las variables con varianza nula, es decir, constantes en todo el conjunto y por tanto sin capacidad discriminativa.
- **Relevancia lineal:** Se eliminan las variables cuya correlación absoluta de Pearson con `class3` es inferior a 0,025.
- **Filtro de redundancia:** Mediante clustering jerárquico sobre la matriz de correlación, se identifican pares de variables altamente correlacionadas entre sí (coeficiente $> 0,97$). En estos casos, ambas vari-

ables contienen información redundante, por lo que se conserva únicamente una de ellas y se elimina la otra, evitando así problemas de multicolinealidad. La Tabla 2.7 recoge los pares más representativos.

Par de características	Correlación
(Avg_num_Proc/s, Std_num_proc/s)	> 0,97
(Succesful_login, Process_activity)	> 0,97
(Succesful_login, is_privileged)	> 0,97
(Process_activity, is_privileged)	> 0,97
(Is_SYN_ACK, is_pure_ack)	> 0,97
(is_syn_only, FIN or RST)	> 0,97
(OSSEC_alert, OSSEC_alert_level)	> 0,97
(paket_rate, byte_rate)	> 0,97
(Login_attempt, Succesful_login)	> 0,97
(Login_attempt, Process_activity)	> 0,97

Table 2.7: Pares de características con correlación superior a 0,97.

2.4.5 Detección de valores atípicos

El conjunto de datos contiene **229.155 filas** consideradas como valores atípicos (*outliers*). Se ha tomado la decisión de **no eliminarlos**: en un sistema de detección de anomalías, muchos de estos valores extremos representan precisamente los comportamientos maliciosos que el modelo debe aprender a identificar. Eliminarlos supondría descartar información crítica y comprometería la capacidad del sistema de generalizar frente a ataques reales.

2.4.6 Análisis del desbalanceo de clases

Tal como se ha descrito en la sección 2.4.2, el dataset presenta un marcado desbalanceo en las variables objetivo de clasificación multiclase. Este desequilibrio es especialmente pronunciado en los tipos de ataque menos frecuentes: Man-in-the-middle (0,03%), Fake notification (0,01%) y Crypto ransomware (0,11%), que pese a su baja representación pueden corresponder a amenazas de alta criticidad. Para mitigar el sesgo hacia las clases mayoritarias durante la evaluación, se utilizará el promediado **MACRO** en todas las métricas de rendimiento. Adicionalmente, en la fase de modelado se explorarán técnicas de balanceo de clases para mejorar el rendimiento sobre las categorías menos representadas.

2.4.7 Consideraciones sobre la calidad de los datos y pipeline final

Durante el análisis se han identificado varios problemas de calidad que deben tratarse antes del modelado. En primer lugar, se han encontrado **valores no válidos** representados mediante los caracteres "-", "?" y

"nan", reemplazados por NaN. En total se han corregido **1.911.396 valores**. Como se observa en la Figura 2.4, las características más afectadas son variables relacionadas con el tráfico de red (byte_rate, total_bytes, Des_bytes, Scr_bytes y paket_rate, Des_bytes_ratio, Scr_bytes_ratio), con porcentajes de nulos próximos al 27%, mientras que Duration presenta en torno al 9%.

Dado que estas variables describen el volumen y comportamiento del tráfico de red, resultan clave para la detección de anomalías. Por ello, no se considera adecuado eliminarlas a pesar de la presencia de valores ausentes. Asimismo, eliminar las instancias con valores nulos supondría una pérdida significativa de información, dado el elevado número de registros afectados, por lo que se opta por un enfoque de imputación que permite conservar tanto las variables como las instancias sin introducir sesgos relevantes en el modelo.

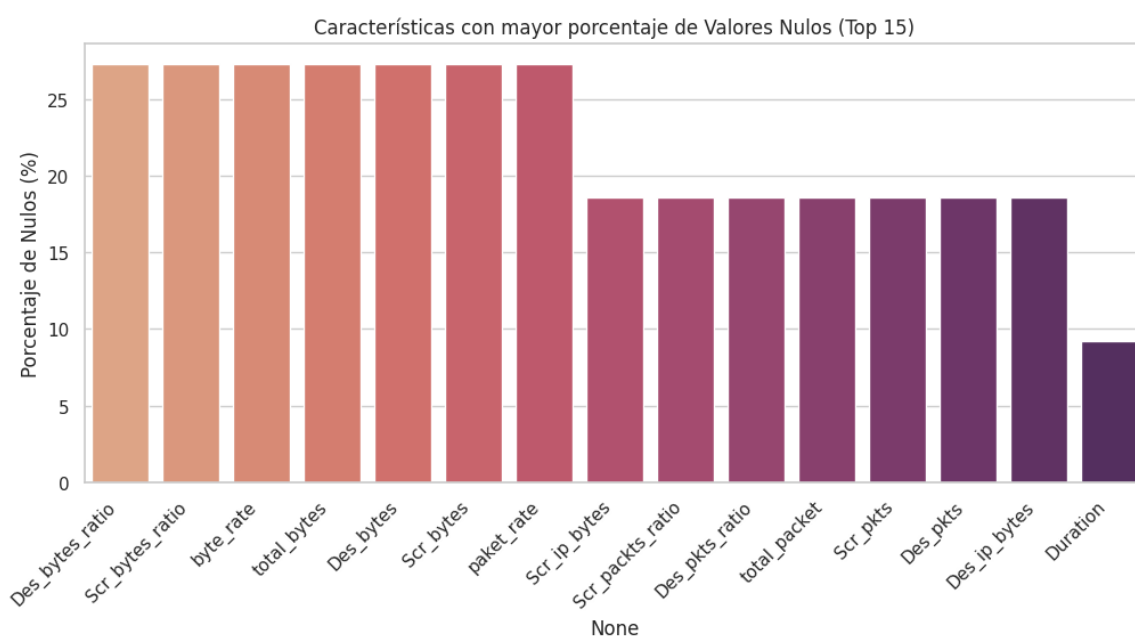


Figure 2.4: Top 15 de características con mayor porcentaje de valores nulos (véase Anexo).

En segundo lugar, la presencia de valores inválidos provocó que muchas características no tuviesen asignado su tipo de dato correcto, lo que fue corregido mediante un script de preprocesamiento.

Por último, el dataset **no dispone de partición predefinida** para entrenamiento y validación. Se ha establecido la siguiente división estratificada, garantizando que todas las transformaciones se ajusten exclusivamente sobre el conjunto de entrenamiento para evitar *data leakage*:

- **Entrenamiento:** 70% — 574.583 registros.
- **Validación:** 15% — 123.125 registros.
- **Prueba:** 15% — 123.126 registros.

Una vez realizada la partición, y a modo de síntesis de lo anteriormente explicado, concluimos con la definición del pipeline de transformación compuesto por los siguientes pasos: **imputación** de nulos numéricos con la media y de categóricos con la etiqueta `missing`, empleando la media en lugar de la mediana para preservar la estructura global de las distribuciones y mantener la coherencia con el posterior uso de **PCA**, que se aplica sobre los datos ya transformados y es sensible a la varianza de las características.

A continuación se aplica un **escalado robusto** mediante `RobustScaler`, cuya elección se fundamenta en que su procedimiento se basa en percentiles, en lugar de emplear la media y la desviación estándar. Asimismo, esta técnica presenta una mayor resistencia frente a valores atípicos, los cuales se han conservado deliberadamente, ya que en su mayoría podrían corresponder a instancias anómalas de interés para el desarrollo de la solución. Y, una vez realizado, aplicamos una **codificación categórica** mediante *One-Hot Encoding*, evitando que los modelos asuman un orden numérico irreal entre categorías.

Tras este proceso de transformación, se aplica el método de **Selección por PCA Feature Subset Selection (PCAFSS)**, con el objetivo de seleccionar las características más relevantes y mantener la mayor cantidad de información posible con un número reducido de variables. Este procedimiento se calcula sobre la totalidad del conjunto de entrenamiento, lo que permite capturar de forma precisa la varianza real del sistema y, en consecuencia, mejorar el rendimiento de los modelos posteriores al reducir la complejidad del problema. Tras su aplicación, **PCA** retiene 7 componentes principales, los cuales explican aproximadamente el 95% de la varianza del conjunto de datos. A partir de estos resultados, identificamos aquellas variables originales del conjunto que presentan una mayor contribución a la formación de dichos componentes y, como resultado, se obtiene una selección final de **34 variables** del conjunto de datos original.

En este contexto, decidimos mantener estas características originales en lugar de emplear directamente las transformaciones generadas por **PCA** con el objetivo de maximizar la interpretabilidad del modelo final en entornos industriales.

2.5 Evaluación del rendimiento. Métricas

2.5.1 Definición de las métricas de evaluación

La evaluación del rendimiento de los modelos constituye una fase fundamental en el desarrollo de sistemas de aprendizaje automático, especialmente en problemas relacionados con la detección de intrusiones en entornos industriales. En este tipo de sistemas, una clasificación incorrecta puede tener consecuencias relevantes: un ataque no detectado puede comprometer la seguridad de la infraestructura, mientras que un número elevado de falsas alarmas puede generar una sobrecarga operativa en los equipos encargados de la supervisión.

Para evaluar el rendimiento de los modelos desarrollados en las fases posteriores del proyecto, se emplearán distintas métricas derivadas de la matriz de confusión. Esta matriz permite analizar la relación entre las predicciones del modelo y las etiquetas reales del conjunto de datos, distinguiendo cuatro tipos de resultados:

- **True Positives (TP):** Número de eventos anómalos correctamente identificados como ataques por el modelo.
- **True Negatives (TN):** Número de eventos normales correctamente clasificados como tráfico legítimo.
- **False Positives (FP):** Eventos normales que el modelo clasifica incorrectamente como ataques, generando falsas alarmas.
- **False Negatives (FN):** Eventos anómalos que el modelo no logra detectar y que son clasificados erróneamente como tráfico normal.

A partir de estos valores se definen las principales métricas utilizadas para evaluar el rendimiento de los modelos de clasificación.

- **Accuracy**

La *accuracy* representa la proporción total de predicciones correctas realizadas por el modelo respecto al número total de observaciones evaluadas.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Aunque proporciona una visión general del rendimiento del modelo, esta métrica puede resultar poco representativa en conjuntos de datos desbalanceados.

- **Precision**

La *precision* mide la proporción de instancias clasificadas como ataques que realmente corresponden a ataques reales.

$$Precision = \frac{TP}{TP + FP}$$

Un valor elevado indica que el modelo genera pocas falsas alarmas.

- **Recall**

El *recall*, también denominado sensibilidad o tasa de verdaderos positivos, mide la proporción de ataques que el modelo logra detectar correctamente.

$$Recall = \frac{TP}{TP + FN}$$

En sistemas de detección de intrusiones esta métrica resulta especialmente relevante, ya que valores bajos implican la existencia de ataques no identificados.

- **F1-score**

El *F1-score* combina precisión y recall mediante su media armónica.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Esta métrica permite evaluar de forma conjunta la capacidad de detección y el control de falsas alarmas.

- **False Positive Rate (FPR)**

La tasa de falsos positivos mide la proporción de eventos normales clasificados incorrectamente como ataques.

$$FPR = \frac{FP}{FP + TN}$$

Valores elevados indican un mayor número de alertas incorrectas generadas por el sistema.

- **AUC-ROC**

La métrica *Area Under the Receiver Operating Characteristic Curve* (AUC-ROC) evalúa la capacidad del modelo para diferenciar entre clases a distintos umbrales de decisión. La curva ROC representa la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos.

Valores cercanos a 1 indican una alta capacidad de discriminación entre tráfico normal y tráfico malicioso.

En conjunto, estas métricas permiten evaluar el rendimiento del modelo desde diferentes perspectivas, considerando tanto la detección de ataques como la generación de errores de clasificación.

2.5.2 Justificación de las métricas seleccionadas

La selección de las métricas de evaluación debe adaptarse al contexto del problema. En sistemas de detección de intrusiones no basta con maximizar la proporción de clasificaciones correctas, sino que es necesario considerar el impacto de los distintos tipos de error.

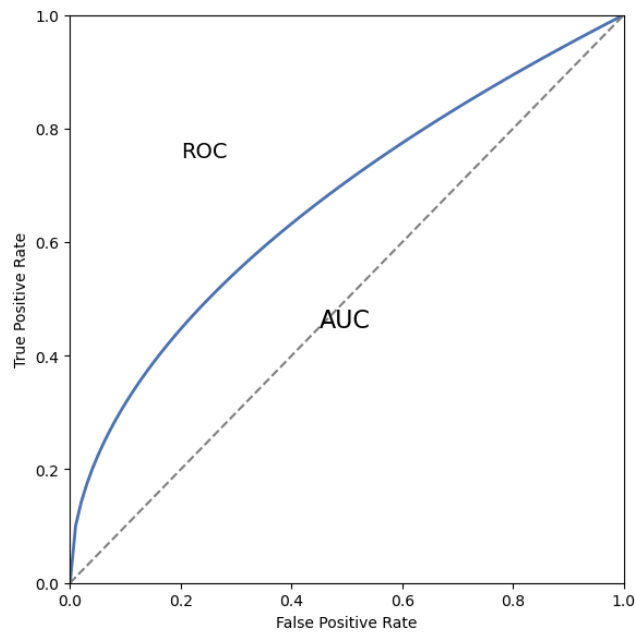


Figure 2.5: Curva ROC que muestra la relación entre TPR y FPR.

El *recall* es especialmente relevante, ya que mide la proporción de ataques correctamente detectados. Un valor reducido implicaría la existencia de intrusiones que el sistema no logra identificar.

Por su parte, la *precision* y la tasa de falsos positivos (*FPR*) permiten analizar el impacto operativo del sistema, ya que un número elevado de falsas alarmas incrementa el volumen de eventos que deben ser revisados manualmente.

El *F1-score* resulta especialmente útil en conjuntos de datos desbalanceados, donde permite evaluar conjuntamente la capacidad de detección y el control de falsas alarmas.

Finalmente, métricas como *accuracy* y *AUC-ROC* ofrecen una visión global del comportamiento del modelo y permiten comparar distintas aproximaciones durante la fase de experimentación.

2.5.3 Relación entre métricas y objetivos del sistema

Las métricas definidas se encuentran alineadas con los objetivos planteados en el Hito 1 del proyecto. En particular, el sistema propuesto busca mejorar la detección de intrusiones y limitar la generación de alertas innecesarias.

El *recall* refleja la capacidad del sistema para detectar intrusiones presentes en el tráfico analizado, mientras que la *precision* y la tasa de falsos positivos (*FPR*) permiten evaluar el impacto de las falsas alarmas generadas por el modelo.

El *F1-score* facilita la comparación entre modelos con distintos comportamientos de clasificación, mientras que métricas como *accuracy* y *AUC-ROC* ofrecen una visión global del rendimiento durante la fase de evaluación.

De este modo, las métricas seleccionadas permiten evaluar tanto el rendimiento técnico del modelo como su impacto en la operación de un sistema de detección de intrusiones en entornos IIoT. Además, en el Hito 1 se introdujeron métricas de impacto operativo, como el número de alertas que requieren revisión manual o el coste asociado a falsos positivos, que permiten analizar las implicaciones prácticas del sistema en un entorno de operación real.

2.6 Conclusiones

En este capítulo se ha presentado el conjunto de datos utilizado en el desarrollo del sistema de detección de intrusiones, describiendo su origen, estructura y características principales. Asimismo, se ha realizado un análisis exploratorio con el objetivo de comprender la distribución de las variables y detectar posibles desequilibrios entre clases.

Además, se han definido las métricas que se emplearán en las fases posteriores para evaluar el rendimiento de los modelos desarrollados.

El análisis realizado establece una base adecuada para las siguientes etapas del trabajo, en las que se abordará el entrenamiento y la evaluación de distintos modelos de aprendizaje automático aplicados a la detección de anomalías en entornos IIoT.

Hito 3: Modelado, experimentación y validación

3.1 Introducción

Tras llevar a cabo un análisis y una preparación exhaustiva del conjunto de datos X-IIoTID, así como una evaluación detallada de su calidad en términos de características, distribuciones y métricas de evaluación más adecuadas, se procede a una de las fases más relevantes del proyecto. En esta etapa, se desarrollará la experimentación y validación del conjunto de datos mediante una selección de modelos de aprendizaje automático. Resultando fundamental, ya que, una vez completada, permitirá avanzar hacia etapas posteriores orientadas al despliegue y la monitorización del sistema en un entorno de producción.

Con el objetivo de obtener los mejores resultados posibles, se llevarán a cabo experimentos con diversos modelos que, para el problema planteado, pueden ofrecer un alto rendimiento. Esta elección se respaldará mediante una selección fundamentada de hiperparámetros —específicos para cada modelo— basada en el conocimiento adquirido previamente y en los análisis derivados de las pruebas realizadas durante el proceso de modelado.

Por otra parte, los análisis finales permitirán validar y demostrar, con un alto grado de consistencia, cuál de los modelos desarrollados obtiene los mejores resultados, así como las razones que lo justifican.

3.2 Modelos

3.2.1 Modelos baseline

Como referencia inicial se incluyeron dos modelos *baseline* con el objetivo de establecer un punto de comparación sencillo frente a modelos de mayor capacidad. En primer lugar, se empleó `LogisticRegression` [11] como *baseline* lineal multiclase. Este modelo resulta especialmente útil en problemas tabulares, ya que permite analizar hasta qué punto la separación entre clases puede aproximarse mediante fronteras lineales. Si su rendimiento queda claramente por debajo del resto de modelos, ello constituye un indicio de la existencia de relaciones no lineales y patrones complejos entre las variables del problema.

En segundo lugar, se utilizó `GaussianNB` [12] como *baseline* probabilístico. Su inclusión responde a dos motivos principales. Por una parte, proporciona una referencia extremadamente rápida y con un coste computacional muy reducido. Por otra, permite evaluar si la hipótesis de independencia condicional entre atributos resulta razonable para este problema. Dado que el conjunto de datos final contiene variables heterogéneas y dependencias entre eventos y tráfico de red, era esperable que este modelo presentase un comportamiento más limitado que los enfoques basados en árboles o las redes neuronales multicapa.

En conjunto, ambos modelos *baseline* no se plantean como candidatos finales, sino como una referencia metodológica que permita interpretar con mayor claridad la mejora obtenida al utilizar modelos no lineales y con una mayor capacidad de representación.

3.2.2 Modelos propuestos

Sobre los *baselines* anteriores se definieron varios modelos propuestos, escogidos con el objetivo de cubrir familias de aprendizaje complementarias y representativas del problema de clasificación directa por tipo. Además, la selección realizada resulta coherente con trabajos recientes en detección de intrusiones y sistemas IIoT, donde los enfoques basados en árboles, *boosting* y redes neuronales han mostrado un comportamiento especialmente competitivo [13] [14] [15].

En primer lugar, se entrenó un `DecisionTree` [16]. Este modelo ofrece una referencia interpretable, rápida y particularmente adecuada para datos tabulares. Además, permite capturar reglas no lineales y relaciones jerárquicas entre variables sin requerir transformaciones complejas adicionales.

A continuación, se utilizaron dos modelos *ensemble* basados en árboles: `RandomForest` [17] y `ExtraTrees`. Ambos enfoques combinan múltiples árboles de decisión para reducir la varianza y mejorar la capacidad de generalización frente a un árbol individual. En particular, `RandomForest` actúa como uno de los principales candidatos de rendimiento debido a su robustez en problemas tabulares con clases desbalanceadas,

mientras que ExtraTrees introduce una aleatorización adicional en la construcción de los árboles con el objetivo de ampliar la comparativa dentro de la familia *tree-based*.

También se incorporó XGBoost [14] como representante de los métodos de *gradient boosting*. Este algoritmo constituye una de las referencias más utilizadas en aprendizaje automático sobre datos tabulares debido a su capacidad para modelar relaciones complejas, controlar el sobreajuste y mantener un rendimiento elevado incluso en conjuntos de datos desbalanceados y de gran tamaño.

Desde la perspectiva neuronal, se incluyeron dos aproximaciones distintas. Por una parte, se entrenó una red neuronal multicapa mediante MLPClassifier [18] [19], utilizada como referencia neuronal clásica integrada en scikit-learn. Por otra parte, se desarrolló una red neuronal profunda adicional (DNN) implementada en PyTorch, incorporando capas densas, normalización por lotes, *dropout* y parada temprana. Esta segunda aproximación permite disponer de una alternativa neuronal más flexible y cercana a arquitecturas profundas utilizadas habitualmente en aprendizaje profundo.

De este modo, la comparativa final queda formada por ocho modelos: DecisionTree, RandomForest, ExtraTrees, XGBoost, LogisticRegression, GaussianNB, MLP y DNN. Esta selección cubre modelos lineales, probabilísticos, basados en árboles, *boosting* y neuronales, proporcionando una base suficientemente representativa para analizar el comportamiento del problema desde distintas familias de aprendizaje.

3.2.3 Justificación

La selección de modelos se justificó a partir de varias características del problema identificadas durante el Hito 2. En primer lugar, el análisis exploratorio realizado previamente mostró la existencia de relaciones no lineales, zonas de solapamiento entre clases y un fuerte desbalanceo entre ataques frecuentes y ataques con muy pocas muestras. Bajo estas condiciones, resultaba necesario comparar modelos simples, utilizados como referencia metodológica, con modelos capaces de capturar interacciones complejas entre variables y fronteras de decisión no lineales.

En segundo lugar, la naturaleza tabular del conjunto de datos hacía especialmente relevante la inclusión de modelos basados en árboles y métodos *ensemble*, ya que este tipo de enfoques suele ofrecer un rendimiento muy competitivo en problemas estructurados de clasificación multiclase. Por este motivo, la comparativa incorpora tanto modelos basados en un único árbol como métodos *bagging* y *boosting*, permitiendo analizar distintos niveles de complejidad dentro de la misma familia de aprendizaje.

Desde la perspectiva neuronal, también resultaba de interés evaluar si una arquitectura profunda era

capaz de competir con los enfoques clásicos sobre este conjunto de datos. Para ello, se incluyeron tanto una red neuronal multicapa integrada en `scikit-learn` como una implementación más flexible desarrollada en `PyTorch`. Esta comparación permite analizar hasta qué punto las arquitecturas neuronales aportan ventajas reales frente a los modelos *tree-based* en un escenario tabular de detección de intrusiones.

Por último, el diseño experimental debía mantener coherencia con la fase posterior de interpretabilidad. Bajo esta restricción, los modelos basados en árboles resultaban especialmente atractivos, ya que combinan un rendimiento elevado en datos tabulares con una interpretación relativamente natural mediante importancia de variables y técnicas *post hoc*. Por esta razón, aunque la comparativa incluye modelos neuronales, los enfoques basados en árboles ocupan un papel central dentro del análisis experimental.

3.3 Configuración experimental

3.3.1 Detalles de implementación

La fase de modelado parte directamente de las salidas generadas en el Hito 2. En concreto, se utilizaron las matrices finales `X_train_final`, `X_val_final` y `X_test_final`, cada una de ellas compuesta por 34 variables finales tras el proceso de selección y preprocesamiento. Las etiquetas se reconstruyeron de forma trazable a partir del conjunto de datos original, respetando exactamente el mismo particionado determinista definido previamente. El problema principal se formuló como una clasificación directa de `class1`; a partir del tipo predicho se derivaron posteriormente `class2` (categoría de ataque) y `class3` (clasificación binaria entre `normal` y `attack`).

La implementación se realizó principalmente en Python utilizando `scikit-learn`, `XGBoost` y `PyTorch`. Todos los modelos se entrenaron íntegramente dentro del notebook correspondiente al Hito 3, y tanto las predicciones como las tablas, figuras, historiales de entrenamiento y modelos serializados se exportaron automáticamente a una carpeta de resultados específica. Esta decisión permite mantener la trazabilidad completa del experimento y evita depender de artefactos calculados fuera del flujo experimental actual.

Todos los modelos se entrenaron bajo el mismo protocolo experimental y utilizando exactamente las mismas particiones de entrenamiento, validación y prueba. Además, se fijó una semilla aleatoria común para garantizar la reproducibilidad de los resultados y reducir la variabilidad entre ejecuciones.

La Tabla 3.1 resume el tamaño de cada partición utilizada durante el proceso de entrenamiento, validación y evaluación final.

La Tabla 3.2 recoge la configuración principal de los modelos finalmente entrenados.

Conjunto	Filas en X	Filas en y	Número de variables
Train	574583	574583	34
Validación	123125	123125	34
Test	123126	123126	34

Table 3.1: Resumen de particiones utilizadas en el Hito 3.

Modelo	Familia	Configuración principal
DecisionTree	Árbol de decisión	<code>class_weight='balanced', max_depth=25, min_samples_leaf=5.</code>
RandomForest	Bosque aleatorio	<code>n_estimators=120, class_weight='balanced_subsample', min_samples_leaf=2.</code>
ExtraTrees	Bosque extremadamente aleatorizado	<code>n_estimators=140, class_weight='balanced', min_samples_leaf=2.</code>
XGBoost	Gradient boosting	<code>tree_method='hist', n_estimators=140, max_depth=8, learning_rate=0.08.</code>
LogisticRegression	Regresión logística	<code>solver='saga', class_weight='balanced', max_iter=140.</code>
GaussianNB	Naive Bayes gaussiano	Configuración por defecto.
MLP	Red neuronal multicapa	<code>hidden_layer_sizes=(128,64,32), early_stopping=True, max_iter=25.</code>
DNN	Red neuronal profunda	<code>Capas densas (256,128,64), dropout=0.25, early_stopping=True.</code>

Table 3.2: Configuración principal de los modelos entrenados en el Hito 3.

3.3.2 Ajuste de hiperparámetros

El ajuste de hiperparámetros se realizó de forma iterativa sobre el conjunto de validación, evaluando manualmente distintas configuraciones razonables para cada familia de modelos. Dado el tamaño del conjunto de entrenamiento y el elevado coste computacional asociado a algunos algoritmos, no se llevó a cabo una búsqueda exhaustiva mediante estrategias de tipo *grid search* sobre todas las combinaciones posibles. En su lugar, se priorizó una búsqueda heurística y acotada centrada en los hiperparámetros con mayor impacto esperado sobre la capacidad de generalización de cada modelo.

Esta decisión resulta coherente con la naturaleza del problema y con recomendaciones habituales en aprendizaje automático aplicado a datos tabulares de gran tamaño. En escenarios con cientos de miles de muestras y múltiples familias de modelos heterogéneas, una optimización exhaustiva puede incrementar considerablemente el coste computacional sin garantizar mejoras proporcionales en rendimiento. Por este motivo, el ajuste se orientó a obtener configuraciones estables, reproducibles y suficientemente representativas para realizar una comparativa experimental sólida entre modelos.

La métrica principal utilizada para comparar configuraciones fue la media simple entre $F1$ binaria, $F1$ por categoría y $F1$ por tipo. Esta decisión evita optimizar exclusivamente la salida binaria, que constituye el nivel más sencillo del problema, y obliga a valorar también la capacidad real del modelo para distinguir entre categorías y tipos concretos de ataque. No obstante, dado que `class2` y `class3` se derivan de forma

determinista a partir de `class1`, la métrica principal del problema continúa siendo el rendimiento obtenido sobre la clasificación directa por tipo.

En los modelos basados en árboles, el ajuste se centró principalmente en controlar la complejidad estructural y limitar el sobreajuste mediante hiperparámetros como profundidad máxima, tamaño mínimo de hoja y número de árboles del ensemble. Además, se utilizaron estrategias de balanceo mediante `class_weight` para mitigar el fuerte desbalanceo entre clases observado durante el análisis exploratorio. En el caso de XGBoost, también se ajustaron parámetros relacionados con la tasa de aprendizaje, el submuestreo y la regularización del proceso de *boosting*. Asimismo, se empleó el método `tree_method='hist'` con el objetivo de reducir el coste computacional del entrenamiento sobre un conjunto de datos de gran tamaño.

En `LogisticRegression` se priorizó la estabilidad del entrenamiento y la convergencia del optimizador mediante el uso del *solver* `saga`, adecuado para problemas multiclase y conjuntos de datos extensos. Por su parte, `GaussianNB` se mantuvo prácticamente sin ajuste al tratarse de un *baseline* probabilístico de referencia.

En los modelos neuronales, el ajuste se centró principalmente en la arquitectura de capas ocultas, el tamaño de lote y los mecanismos de regularización. En MLP se utilizó parada temprana para evitar sobreentrenamiento innecesario, mientras que en la DNN implementada en PyTorch se incorporaron técnicas adicionales como *dropout*, normalización por lotes (*batch normalization*) y parada temprana basada en el rendimiento sobre validación.

El objetivo de esta fase no fue obtener la configuración óptima absoluta para cada modelo de manera aislada, sino construir una comparativa coherente, reproducible y suficientemente representativa entre distintas familias de aprendizaje automático.

3.3.3 Espacio de búsqueda

El espacio de búsqueda considerado fue deliberadamente reducido y se centró en los hiperparámetros con mayor impacto esperado dentro de cada familia de modelos:

- **DecisionTree:** profundidad máxima, número mínimo de ejemplos por hoja y uso de pesos de clase.
- **RandomForest:** número de árboles, tamaño mínimo de hoja y estrategia de balanceo por clase.
- **ExtraTrees:** número de árboles, tamaño mínimo de hoja y uso de pesos de clase para controlar el desbalanceo.

- **XGBoost:** número de rondas de *boosting*, profundidad máxima, *learning rate*, submuestreo y método de construcción de árboles.
- **LogisticRegression:** tipo de *solver*, número máximo de iteraciones y uso de pesos de clase.
- **GaussianNB:** configuración simple sin ajuste intensivo, al tratarse de un *baseline* de referencia.
- **MLP:** estructura de capas ocultas, criterio de parada temprana, número máximo de iteraciones y tamaño de lote.
- **DNN:** número y tamaño de capas densas, *dropout*, tasa de aprendizaje, tamaño de lote y criterio de parada temprana.

Este diseño del espacio de búsqueda resulta coherente con el objetivo general del trabajo: construir una comparativa trazable y reproducible entre distintas familias de modelos, priorizando el análisis experimental y la interpretabilidad de los resultados frente a una optimización extrema e independiente de cada algoritmo.

3.4 Resultados

Los resultados de los modelos han sido obtenidos con los recursos hardware que se reflejan en el **anexo E**.

3.4.1 Resultados globales en train, validación y test

La comparativa global muestra una superioridad clara de los modelos basados en árboles sobre el resto de enfoques considerados. En particular, `RandomForest` obtiene el mejor rendimiento agregado sobre el conjunto de prueba, alcanzando una media simple de $F1$ igual a 0.9777. A continuación se sitúan `XGBoost`, `ExtraTrees` y `MLP`, todos ellos con resultados elevados y relativamente próximos entre sí. En el extremo opuesto, los modelos lineales y probabilísticos presentan un comportamiento considerablemente inferior, especialmente en la clasificación directa por tipo.

Además del rendimiento final en test, se analizaron también las métricas obtenidas sobre entrenamiento y validación con el objetivo de estudiar la capacidad de generalización de cada modelo. Esta comparación permite observar no solo qué modelos obtienen mejores resultados finales, sino también cómo evoluciona su comportamiento entre particiones y hasta qué punto aparecen síntomas de sobreajuste.

La Tabla 3.3 resume el rendimiento global obtenido en entrenamiento, validación y prueba mediante la media simple entre $F1$ binaria, $F1$ por categoría y $F1$ por tipo, junto con el tiempo aproximado de entrenamiento de cada modelo.

Modelo	$F1_{train}$	$F1_{val}$	$F1_{test}$	Tiempo (s)
RandomForest	0.9981	0.9722	0.9777	148.74
XGBoost	0.9640	0.9543	0.9583	215.86
ExtraTrees	0.9718	0.9515	0.9559	81.61
MLP	0.9569	0.9477	0.9546	72.27
DecisionTree	0.9606	0.9458	0.9529	10.67
DNN	0.7658	0.7667	0.7643	150.71
GaussianNB	0.5987	0.5968	0.5969	0.23
LogisticRegression	0.5379	0.5384	0.5361	156.30

Table 3.3: Resultados globales en entrenamiento, validación y test.

La Figura 3.1 permite comparar visualmente la evolución de la media de $F1$ entre entrenamiento, validación y test. En los modelos con mejor rendimiento, las diferencias entre validación y prueba permanecen relativamente reducidas, lo que sugiere una capacidad de generalización adecuada sobre datos no vistos.

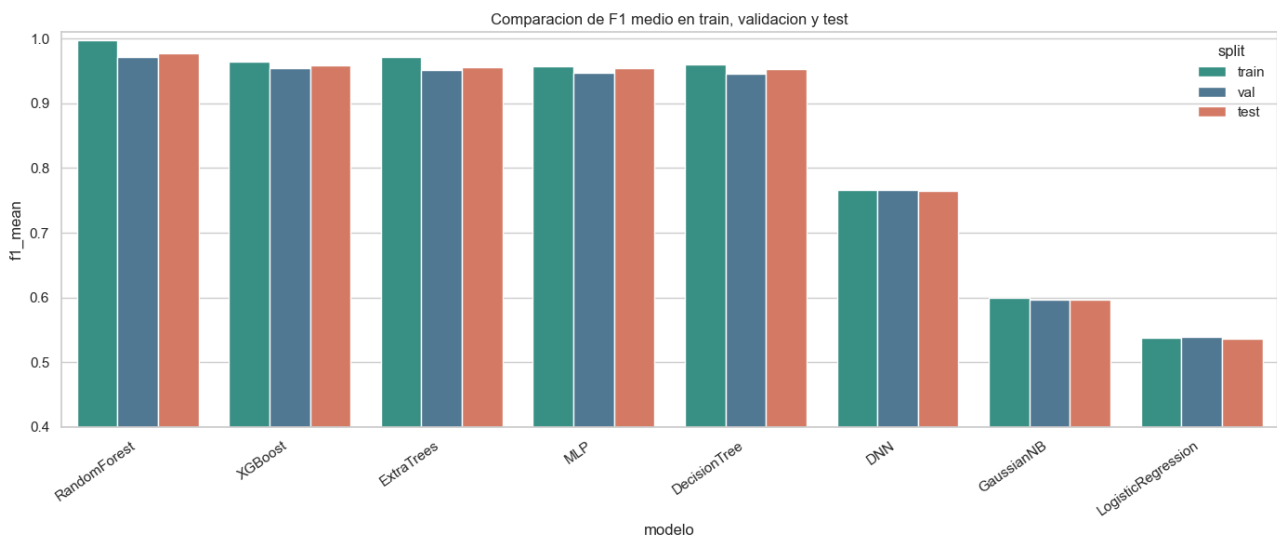


Figure 3.1: Comparativa de la media de $F1$ en entrenamiento, validación y test.

En el caso de RandomForest, puede observarse un rendimiento prácticamente perfecto sobre entrenamiento y un descenso moderado en validación y test. Este comportamiento resulta coherente con la presencia de cierto sobreajuste esperable en modelos *ensemble* basados en árboles, aunque manteniendo un comportamiento muy sólido fuera del conjunto de entrenamiento.

Por el contrario, la DNN presenta un rendimiento significativamente inferior al obtenido por los modelos basados en árboles. Este resultado coincide con observaciones habituales en problemas tabulares estructurados, donde los enfoques *tree-based* suelen superar a arquitecturas neuronales profundas generalistas.

La Figura 3.2 complementa este análisis mostrando el *gap* de generalización entre entrenamiento, validación y test para cada modelo. Valores reducidos indican una mayor estabilidad entre particiones y, por tanto, una mejor capacidad de generalización. En este sentido, los modelos con mejor rendimiento mantienen diferencias moderadas entre entrenamiento y evaluación externa, mientras que los modelos

más simples presentan resultados bajos de forma consistente en todas las particiones.

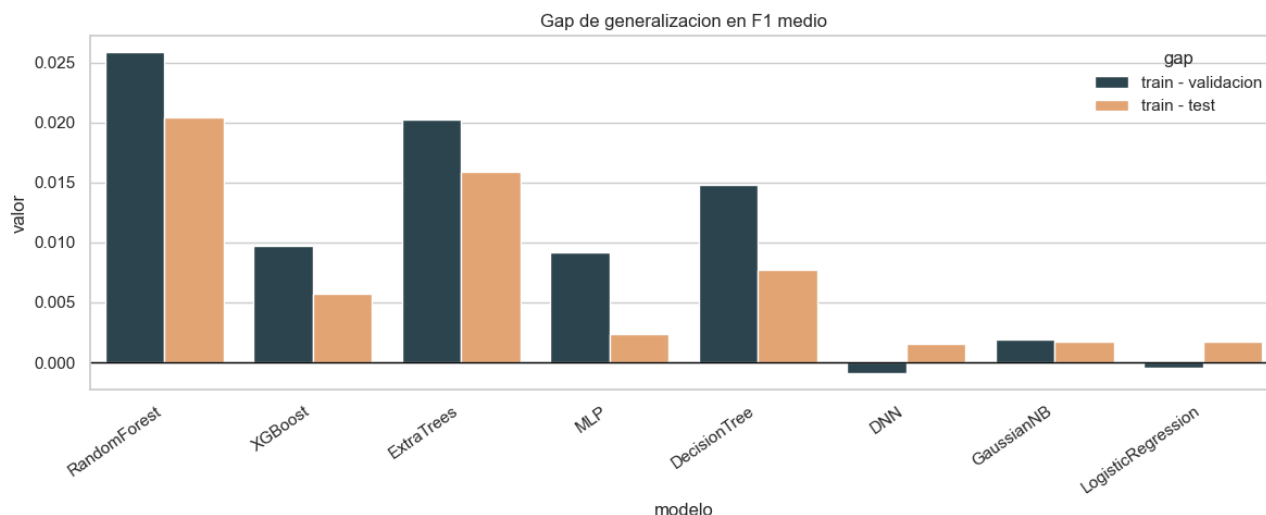


Figure 3.2: Gap de generalización entre entrenamiento, validación y test.

3.4.2 Análisis comparativo entre familias de modelos

La comparativa experimental permite observar diferencias claras entre las distintas familias de modelos consideradas. En términos generales, los enfoques basados en árboles y métodos *ensemble* son los que mejor se adaptan al problema de clasificación planteado. Tanto RandomForest como XGBoost y ExtraTrees obtienen resultados elevados y estables en las distintas particiones, lo que sugiere una buena capacidad para capturar relaciones no lineales y manejar el fuerte desbalanceo existente entre clases.

Dentro de esta familia, RandomForest obtiene el mejor equilibrio global entre rendimiento y estabilidad. Por su parte, XGBoost alcanza resultados muy próximos, aunque con un coste computacional superior asociado al proceso iterativo de *boosting*. ExtraTrees también mantiene un comportamiento competitivo, mostrando que la aleatorización adicional introducida en la construcción de los árboles no perjudica significativamente la capacidad de generalización del modelo.

El árbol individual (DecisionTree) presenta un rendimiento notablemente alto pese a su simplicidad relativa y su bajo tiempo de entrenamiento. Este comportamiento resulta especialmente relevante desde el punto de vista interpretativo, ya que demuestra que una parte importante de la estructura del problema puede capturarse mediante reglas jerárquicas relativamente sencillas.

Desde la perspectiva neuronal, MLP mantiene un rendimiento competitivo y relativamente cercano al bloque principal de modelos. Sin embargo, la DNN implementada en PyTorch obtiene resultados considerablemente inferiores. Este comportamiento coincide con observaciones habituales en problemas tabulares

estructurados, donde las arquitecturas profundas generalistas suelen quedar por detrás de los modelos *tree-based*, especialmente cuando el número de variables es moderado y existen fuertes diferencias de soporte entre clases.

Por último, los modelos *baseline* (LogisticRegression y GaussianNB) muestran que el problema no puede abordarse adecuadamente mediante hipótesis excesivamente simples. Ni la separación lineal implícita en la regresión logística ni la independencia condicional asumida por Naive Bayes consiguen representar correctamente la complejidad multiclase presente en el conjunto de datos.

3.4.3 Dinámica de aprendizaje

Además de las métricas finales obtenidas en validación y test, se analizaron las curvas de aprendizaje de aquellos modelos que exponen de forma natural información sobre la evolución del entrenamiento. Este análisis permite estudiar la estabilidad del proceso de optimización, detectar posibles síntomas de sobreajuste y observar cómo evoluciona la capacidad de generalización a lo largo del entrenamiento.

En el caso de la DNN, la Figura 3.3 muestra la evolución de la pérdida y del *F1* macro por tipo tanto en entrenamiento como en validación. Puede observarse que el modelo converge rápidamente durante las primeras épocas y posteriormente tiende a estabilizarse. Sin embargo, el rendimiento alcanzado queda claramente por debajo del obtenido por los mejores modelos basados en árboles.

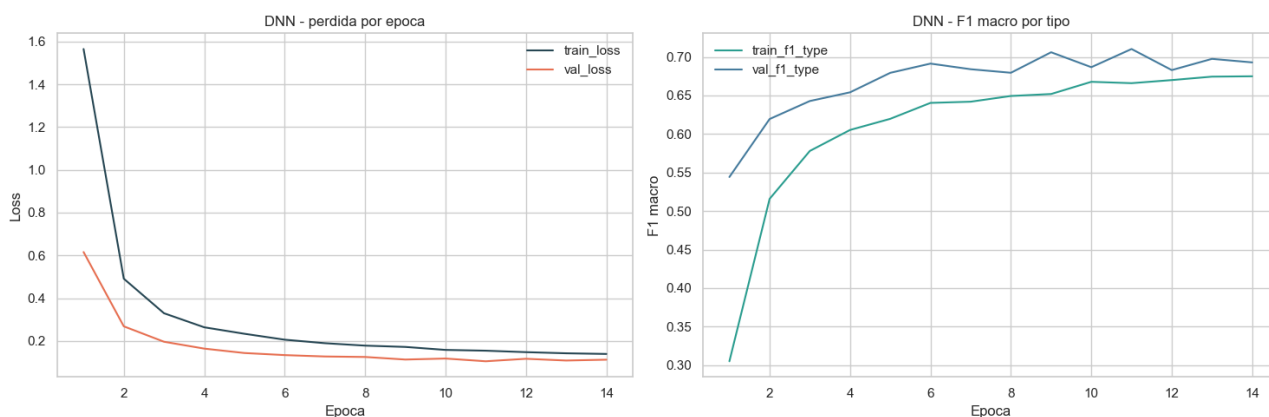


Figure 3.3: Evolución de la pérdida y del *F1* macro durante el entrenamiento de la DNN.

Por su parte, XGBoost presenta un comportamiento estable durante las distintas rondas de *boosting*. La Figura 3.4 muestra cómo la métrica *mlogloss* disminuye progresivamente tanto en entrenamiento como en validación sin divergencias bruscas entre ambas curvas, lo que sugiere un proceso de aprendizaje relativamente controlado.

En MLP, la evolución de la pérdida y de la puntuación interna de validación también refleja una convergencia razonablemente estable. La Figura 3.5 muestra cómo el modelo mejora progresivamente du-

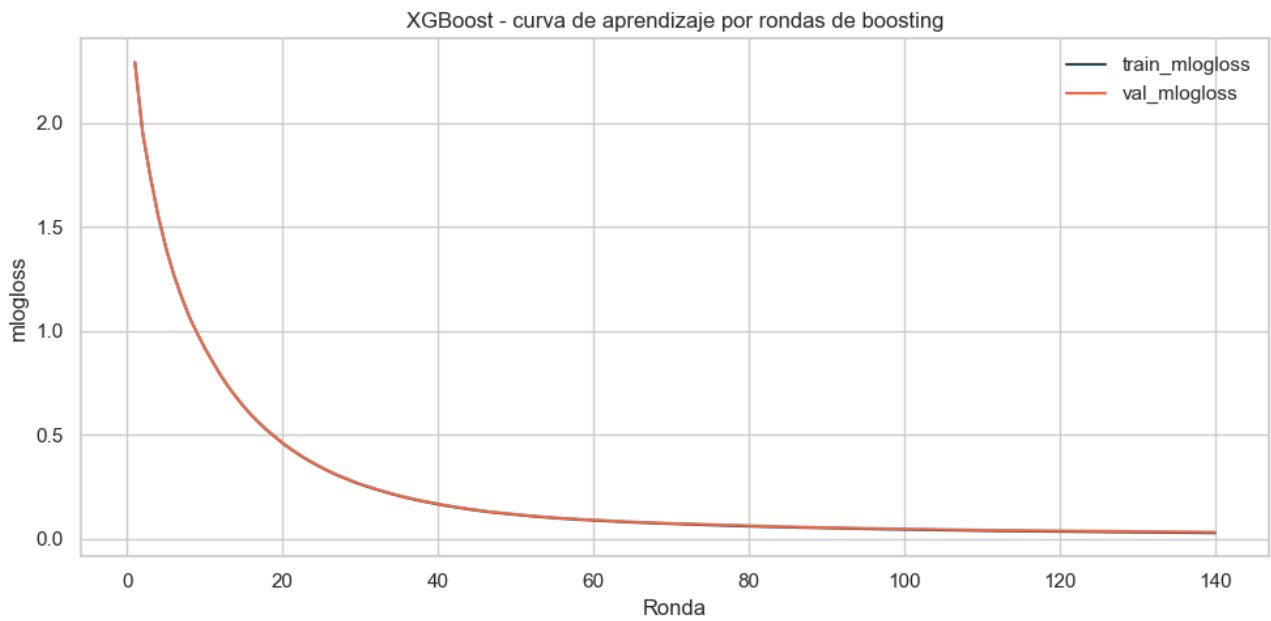


Figure 3.4: Curva de aprendizaje de XGBoost mediante mlogloss en entrenamiento y validación.

durante las iteraciones iniciales hasta alcanzar una zona de estabilización, sin oscilaciones bruscas entre entrenamiento y validación. La utilización de mecanismos de parada temprana permitió además limitar el sobreentrenamiento y mantener un equilibrio adecuado entre rendimiento y coste computacional.

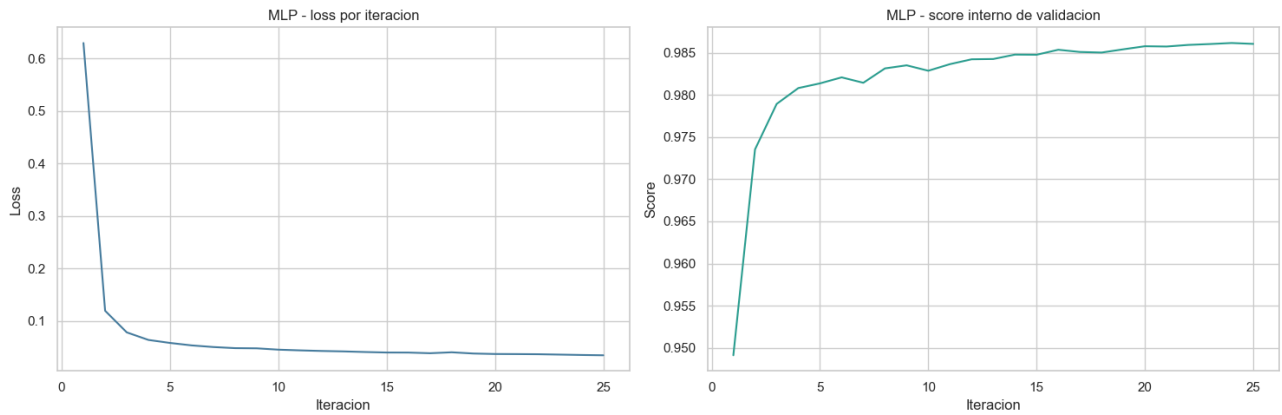


Figure 3.5: Evolución de la pérdida y de la puntuación de validación interna para MLP.

En conjunto, las curvas de aprendizaje refuerzan las conclusiones observadas previamente en las métricas globales. Los modelos con mejor rendimiento muestran procesos de entrenamiento relativamente estables y sin síntomas extremos de sobreajuste, mientras que las arquitecturas neuronales profundas no consiguen alcanzar el mismo nivel de rendimiento que los enfoques *tree-based* sobre este problema tabular.

3.4.4 Resultados finales en test y métricas operativas

La evaluación final sobre el conjunto de prueba confirma que la tarea más sencilla corresponde a la clasificación binaria entre tráfico normal y malicioso, mientras que la clasificación directa por tipo constituye el nivel más exigente del problema. En el mejor modelo, RandomForest, el $F1$ binario alcanza 0.9960, mientras que el $F1$ por categoría y el $F1$ por tipo descienden hasta 0.9786 y 0.9583 respectivamente. Esta diferencia entre niveles de salida resulta coherente con la propia jerarquía del problema: distinguir tráfico benigno frente a tráfico malicioso es considerablemente más sencillo que discriminar entre ataques específicos con bajo soporte y patrones parcialmente solapados.

La Tabla 3.4 resume las principales métricas técnicas obtenidas en test para todos los modelos entrenados. Además de las métricas basadas en $F1$, también se incluyen el área bajo la curva ROC ($AUC-ROC$) y la tasa de falsos positivos (*False Positive Rate*, FPR), especialmente relevantes en problemas de detección de intrusiones.

Modelo	$F1_{bin}$	$F1_{cat}$	$F1_{type}$	Media	AUC	FPR
RandomForest	0.9960	0.9786	0.9583	0.9777	0.9999	0.0022
XGBoost	0.9887	0.9575	0.9288	0.9583	0.9999	0.0208
ExtraTrees	0.9903	0.9532	0.9242	0.9559	0.9997	0.0155
MLP	0.9875	0.9536	0.9225	0.9546	0.9991	0.0065
DecisionTree	0.9863	0.9577	0.9146	0.9529	0.9984	0.0243
DNN	0.8502	0.7357	0.7070	0.7643	0.9946	0.3316
GaussianNB	0.6864	0.5410	0.5634	0.5969	0.5835	0.8626
LogisticRegression	0.6843	0.4685	0.4555	0.5361	0.8945	0.8709

Table 3.4: Resultados finales en test.

Desde una perspectiva operativa, también se analizaron métricas relacionadas con el impacto real sobre un equipo de seguridad. Para ello, se estimó el volumen de alertas manuales generadas, el número de ataques perdidos, las horas aproximadas de revisión y el coste operativo derivado de los falsos positivos.

La Tabla 3.5 muestra que RandomForest no solo obtiene el mejor rendimiento técnico global, sino que además mantiene una tasa de falsos positivos especialmente reducida. Esto se traduce en un menor coste asociado a la revisión manual de alertas y en un comportamiento más equilibrado desde el punto de vista operativo.

Modelo	Alertas/1000	Ataques perdidos	Horas revisión	Coste FP (€)
RandomForest	485.04	333	2986.05	176.25
XGBoost	496.80	57	3058.45	1641.25
ExtraTrees	493.03	187	3035.25	1223.75
MLP	481.21	1074	2962.45	512.50
DecisionTree	498.10	123	3066.45	1923.75
DNN	655.98	108	4038.40	26203.75

Table 3.5: Métricas operativas sobre el conjunto de prueba.

La Figura 3.6 compara las curvas ROC binarias de los tres modelos con mejor rendimiento global. Los

resultados muestran un comportamiento muy sólido en todos ellos, con áreas bajo la curva próximas a 1.0 y una clara superioridad frente al resto de enfoques evaluados.

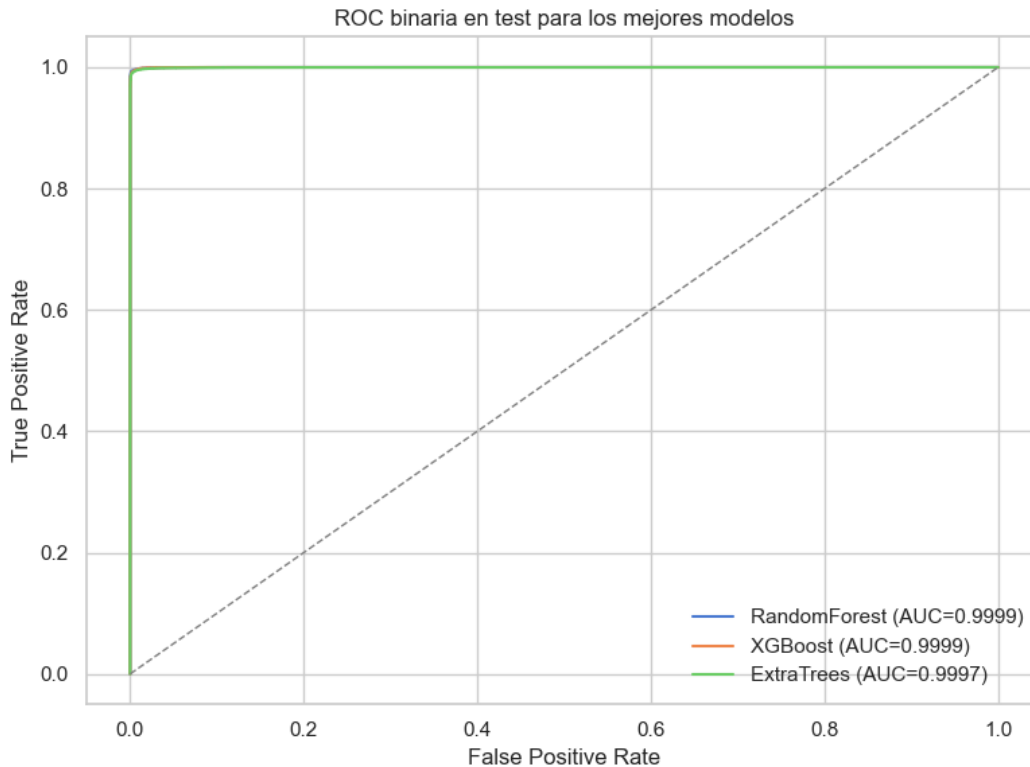


Figure 3.6: Curvas ROC binarias en test para los mejores modelos.

3.4.5 Análisis detallado de los mejores modelos

Con el objetivo de estudiar con mayor detalle el comportamiento de los modelos con mejor rendimiento, se analizaron específicamente RandomForest, XGBoost y ExtraTrees, que constituyen el bloque principal de la comparativa global.

Entre ellos, RandomForest obtiene el mejor equilibrio global entre capacidad de detección y estabilidad operativa, alcanzando simultáneamente la mayor media de $F1$ y la menor tasa de falsos positivos. XGBoost presenta resultados muy próximos en términos de AUC-ROC y capacidad de clasificación, aunque con un mayor coste computacional y una FPR más elevada. Por su parte, ExtraTrees mantiene un comportamiento competitivo y confirma la robustez general de los enfoques basados en árboles sobre este problema tabular.

El análisis por clases muestra que los mayores errores residuales se concentran principalmente en ataques minoritarios o parcialmente solapados respecto a otras categorías. Entre las clases con peor comportamiento medio destacan mitm, fuzzing, tcp relay y reverse_shell, todas ellas con un soporte considerablemente inferior al de las clases mayoritarias del conjunto de datos.

La Figura 3.7 resume el *recall* obtenido por los mejores modelos en estas clases complejas. Puede observarse que, aunque el rendimiento global permanece elevado, siguen existiendo diferencias apreciables entre modelos en ataques específicos de bajo soporte, especialmente en mitm y fuzzing.

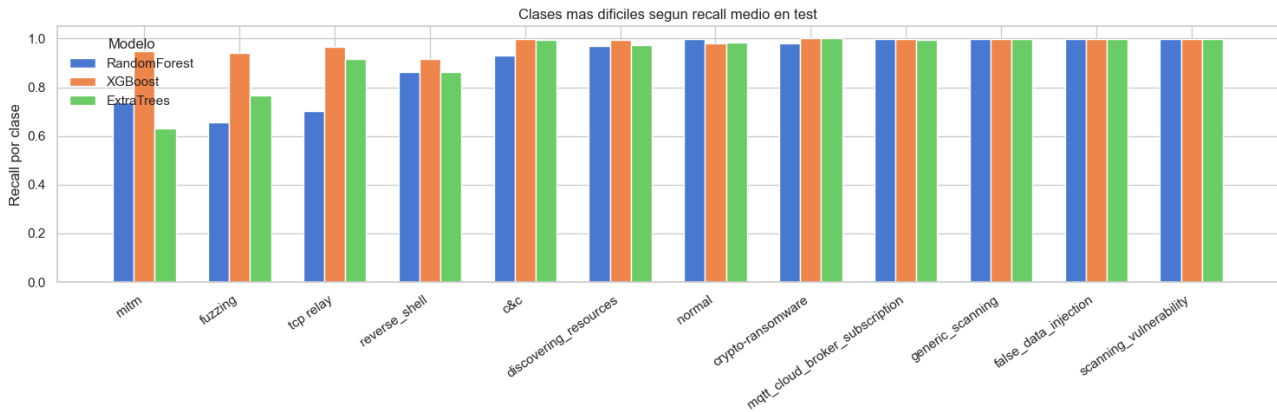


Figure 3.7: Clases más difíciles según el *recall* medio de los mejores modelos.

3.5 Discusión de resultados

3.5.1 Análisis del rendimiento

Los resultados obtenidos sugieren que el problema estudiado presenta una estructura fuertemente no lineal y dependiente de interacciones complejas entre variables. Esta característica explica el bajo rendimiento relativo de los modelos lineales y probabilísticos, cuyos supuestos de separación lineal o independencia condicional resultan demasiado restrictivos para representar adecuadamente el comportamiento del tráfico de red y de los distintos tipos de ataque.

Por el contrario, los modelos basados en árboles muestran una capacidad mucho mayor para capturar relaciones jerárquicas, umbrales y dependencias no lineales entre atributos. Este comportamiento resulta coherente con lo observado habitualmente en problemas tabulares estructurados, donde los métodos *ensemble* basados en árboles suelen ofrecer una combinación especialmente sólida entre capacidad predictiva, estabilidad y robustez frente al desbalanceo.

La mejora observada al pasar de un árbol individual a un modelo ensamblado como RandomForest también sugiere que una parte importante del error del árbol simple está asociada a problemas de varianza. El mecanismo de agregación del bosque aleatorio permite reducir esta sensibilidad y obtener fronteras de decisión más estables sin perder la capacidad de modelar relaciones complejas entre variables.

3.5.2 Discusión sobre clases y casos complejos

El análisis por clases muestra que las mayores dificultades no se concentran en la clase normal, sino en varios ataques minoritarios y parcialmente solapados respecto a otras categorías. Entre los casos más complejos aparecen ataques como `mitm`, `fuzzing`, `tcp relay` o `reverse_shell`, todos ellos caracterizados por un soporte reducido y por patrones de tráfico menos diferenciados que los presentes en las clases mayoritarias.

En estos escenarios aparecen diferencias interesantes entre modelos. Algunos enfoques más agresivos, como el árbol individual, consiguen recuperar mejor determinadas clases minoritarias, aunque a costa de introducir un mayor número de falsos positivos y reducir la precisión global. Por el contrario, modelos ensamblados como `RandomForest` tienden a mantener un equilibrio más estable entre precisión y capacidad de detección, incluso cuando no maximizan el *recall* absoluto en todas las clases.

Este comportamiento pone de manifiesto que la evaluación del problema no puede limitarse únicamente a métricas agregadas globales. El análisis detallado por clase resulta esencial para identificar qué tipos de ataques siguen siendo especialmente problemáticos y cómo afectan las decisiones de modelado al equilibrio entre detección y robustez operativa.

3.5.3 Impacto del desbalanceo

El fuerte desbalanceo entre clases constituye uno de los principales factores que condicionan el comportamiento de los modelos. Mientras que algunas categorías cuentan con decenas de miles de ejemplos, ciertos ataques minoritarios apenas disponen de unas pocas muestras dentro del conjunto de prueba. Esta diferencia de varios órdenes de magnitud dificulta especialmente la capacidad de generalización sobre ataques poco representados.

En este contexto, métricas agregadas tradicionales como la exactitud global o el *F1* ponderado pueden ocultar degradaciones importantes sobre clases minoritarias. Por este motivo, el análisis se centró principalmente en métricas macro y en el estudio individualizado por clase, lo que permitió detectar diferencias relevantes entre modelos que no serían visibles mediante una evaluación puramente global.

La utilización de pesos de clase ayudó a mitigar parcialmente este problema en varios modelos, aunque las dificultades asociadas a ataques poco representados continúan siendo visibles incluso en los enfoques con mejor rendimiento. En consecuencia, el desbalanceo sigue siendo uno de los factores que mejor explica la dificultad relativa observada en determinadas categorías.

3.5.4 Análisis de errores

El análisis de errores muestra que la mayor parte de las confusiones residuales aparecen entre categorías con patrones parcialmente similares o con menor representación dentro del conjunto de entrenamiento. Aunque la separación binaria entre tráfico benigno y malicioso queda prácticamente resuelta en los mejores modelos, siguen existiendo dificultades en determinadas categorías específicas relacionadas con explotación, reconocimiento o movimiento lateral.

También se observa una diferencia clara en el comportamiento de los distintos enfoques frente al compromiso entre sensibilidad y falsos positivos. Los modelos más simples o más agresivos tienden a sobre-detectar ciertos patrones de ataque, recuperando mejor algunas clases minoritarias pero introduciendo un mayor volumen de errores sobre tráfico legítimo. Por el contrario, los métodos ensamblados mantienen fronteras de decisión más estables y un comportamiento global más equilibrado.

3.5.5 Generalización

La comparación entre entrenamiento, validación y test muestra un comportamiento relativamente estable en los modelos con mejor rendimiento. No se observan cambios bruscos de jerarquía entre particiones ni degradaciones severas en evaluación externa, lo que sugiere que las configuraciones seleccionadas mantienen una capacidad de generalización adecuada sobre datos no vistos.

Aunque algunos modelos ensamblados presentan un cierto grado de sobreajuste esperable sobre entrenamiento, este efecto permanece controlado y no compromete el comportamiento final sobre validación y prueba.

3.5.6 Comparación entre modelos

La comparativa final entre modelos permite extraer varias conclusiones relevantes. En primer lugar, los resultados muestran que los modelos con mayor capacidad para representar relaciones no lineales y dependencias complejas entre atributos son también los que alcanzan un comportamiento más estable y robusto en este problema. Este comportamiento resulta coherente con lo observado habitualmente en datos tabulares estructurados, donde los métodos *ensemble* suelen ofrecer ventajas significativas frente a aproximaciones lineales o arquitecturas neuronales generalistas.

En segundo lugar, la red neuronal multicapa (MLP) demuestra que los enfoques neuronales pueden ofrecer resultados competitivos en este escenario, aunque sin llegar a superar a los mejores modelos basados en árboles. La DNN, por su parte, presenta un comportamiento considerablemente más limitado, lo que sugiere que arquitecturas profundas genéricas no siempre resultan la opción más adecuada en problemas

tabulares de este tipo.

Por último, los modelos baseline confirman que una aproximación excesivamente simple resulta insuficiente para representar adecuadamente la complejidad del problema. Tanto la hipótesis lineal de `LogisticRegression` como la independencia condicional asumida por `GaussianNB` limitan significativamente su capacidad para discriminar entre ataques con patrones parcialmente solapados y relaciones no lineales entre variables.

En conjunto, `RandomForest` se consolida como la solución más equilibrada del Hito 3, combinando un rendimiento elevado, buena estabilidad entre particiones y un comportamiento especialmente robusto frente al desbalanceo y las dificultades propias de la clasificación multiclase.

3.6 Interpretabilidad

3.6.1 Importancia de variables

La interpretabilidad del modelo constituye un aspecto fundamental en sistemas de detección de intrusiones en entornos IIoT, ya que permite comprender qué variables influyen en la identificación de comportamientos anómalos y facilita la toma de decisiones por parte de los equipos de seguridad.

En este trabajo, la interpretabilidad se ha abordado principalmente mediante el análisis de la importancia de variables utilizando modelos basados en árboles, concretamente `Random Forest`. Este tipo de modelos permite estimar la contribución relativa de cada característica en el proceso de clasificación.

Los resultados muestran que las variables más relevantes están relacionadas tanto con el comportamiento del tráfico de red como con el estado interno del sistema. En particular, destacan `Duration`, `total_bytes`, `read_write_physical.process` y `Avg_num_cswch/s`, lo que indica que la detección de anomalías no depende de una única señal, sino de la combinación de múltiples indicadores.

Este comportamiento es coherente con la naturaleza del problema, donde los ataques no suelen manifestarse a través de una única variable, sino mediante patrones complejos que combinan cambios en el tráfico de red y en el comportamiento del sistema.

Además, se observa que algunas variables con baja correlación lineal respecto a la variable objetivo presentan una alta importancia en el modelo. Este hecho pone de manifiesto la existencia de relaciones no lineales, que no pueden ser capturadas mediante técnicas estadísticas clásicas, pero sí mediante modelos de aprendizaje automático.

3.6.2 Relación con el EDA

El análisis de interpretabilidad está estrechamente relacionado con los resultados obtenidos durante el análisis exploratorio de datos.

En primer lugar, el EDA evidenció un fuerte solapamiento entre las distribuciones de las variables para tráfico normal y anómalo, lo que indicaba la ausencia de separabilidad lineal. Este resultado se confirma en el comportamiento del modelo, que requiere combinar múltiples variables para realizar una clasificación efectiva. Asimismo, el análisis de correlación mostró que ninguna variable individual presenta una relación lineal fuerte con la variable objetivo. Sin embargo, el modelo es capaz de identificar variables relevantes que, aunque individualmente no sean discriminativas, aportan información significativa cuando se consideran de forma conjunta.

Por otro lado, el proceso de selección de características, basado en criterios de varianza, correlación y redundancia, ha permitido reducir la dimensionalidad del problema manteniendo las variables más informativas. La coherencia entre estas decisiones y las variables finalmente utilizadas por el modelo refuerza la validez del enfoque adoptado.

En conjunto, la interpretabilidad no solo permite comprender el comportamiento del modelo, sino que también valida las conclusiones obtenidas durante el análisis exploratorio y justifica el uso de modelos no lineales para este problema.

3.6.3 Análisis avanzado de interpretabilidad (SHAP/LIME)

Como complemento al análisis global de importancia de variables, se han considerado técnicas avanzadas de interpretabilidad como SHAP (SHapley Additive exPlanations) [20] y LIME (Local Interpretable Model-agnostic Explanations) [21].

Estas técnicas permiten analizar la contribución de cada variable a nivel individual en una predicción concreta, proporcionando una interpretación local del comportamiento del modelo. En particular, SHAP se basa en la teoría de juegos para asignar una contribución a cada característica, mientras que LIME aproxima localmente el modelo mediante una representación interpretable.

Aunque no se han aplicado de forma exhaustiva en este trabajo, su uso permite profundizar en la comprensión del modelo en predicciones individuales.

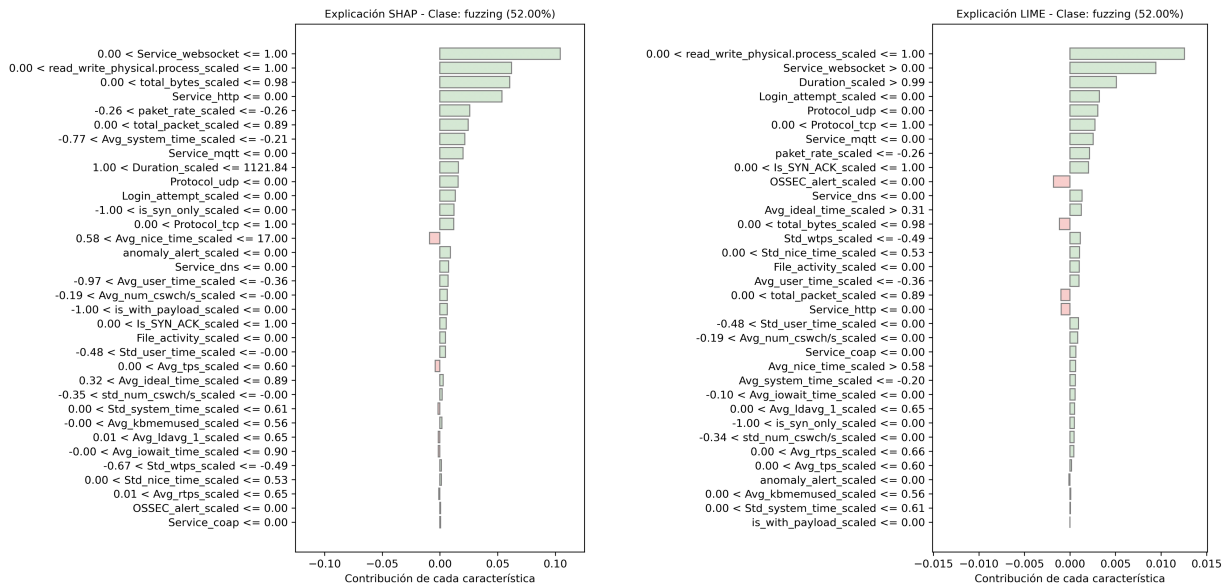


Figure 3.8: Explicación de una instancia mediante SHAP y LIME.

En la Figura 3.8 se analiza una instancia clasificada como un ataque de tipo *fuzzing* [22]. Ambas técnicas coinciden en identificar las variables más influyentes en la predicción.

Entre ellas, destaca `read_write_physical.process_scaled`, asociada a actividad de lectura y escritura a nivel del sistema, lo que puede indicar accesos no habituales. Este comportamiento es coherente con los ataques de tipo *fuzzing*, basados en la generación de entradas inesperadas que provocan interacciones anómalas con los recursos del sistema. Por otro lado, LIME resalta `anomaly_alert_scaled`, lo que sugiere indicios previos de comportamiento anómalo. En paralelo, SHAP identifica variables como `Service_websocket` y `Duration_scaled`, asociadas a patrones de tráfico prolongados.

En conjunto, estas evidencias permiten interpretar la decisión del modelo, reforzando la validez de la predicción y mejorando su transparencia en entornos IIoT.

3.7 Conclusiones

En este hito se ha abordado el desarrollo, ajuste y evaluación de distintos modelos de aprendizaje automático y profundo para la detección de intrusiones en entornos IIoT. Los resultados obtenidos muestran que los enfoques basados en árboles, especialmente los métodos ensamblados, ofrecen el comportamiento más robusto y estable en este problema, superando claramente a los modelos lineales y probabilísticos tradicionales.

El análisis realizado pone de manifiesto que la complejidad del conjunto de datos, caracterizada por relaciones no lineales, solapamiento entre clases y un fuerte desbalanceo, requiere modelos capaces de capturar dependencias complejas entre variables. Asimismo, el estudio de interpretabilidad permitió identificar las características con mayor influencia en las predicciones y verificar la coherencia entre el comportamiento de los modelos y el análisis exploratorio previo.

A pesar de los buenos resultados globales, continúan existiendo dificultades relevantes en determinadas clases minoritarias y ataques poco representados, especialmente en escenarios donde aparecen patrones ambiguos o parcialmente solapados. Este comportamiento confirma que las métricas agregadas deben complementarse con análisis detallados por clase y por categoría para evaluar correctamente la capacidad real de detección.

En conjunto, este hito demuestra la viabilidad de construir sistemas de detección eficaces y razonablemente interpretables para entornos IIoT, destacando la importancia de combinar análisis exploratorio, selección de variables, estrategias de balanceo y evaluación detallada durante todo el proceso de modelado.

Hito 4: Despliegue y paso a producción del modelo

4.1 Objetivo del despliegue

El objetivo de este hito es transformar el sistema de detección de intrusiones desarrollado en los hitos anteriores en una solución funcional, desplegable y accesible mediante servicios independientes.

En el Hito 3 se evaluaron múltiples modelos de aprendizaje automático, identificando distintas alternativas válidas en función de su rendimiento, robustez y comportamiento frente al desbalanceo y la complejidad no lineal del problema. A partir de este análisis, el presente hito se centra en la operacionalización del sistema completo, permitiendo utilizar, versionar y seleccionar distintos modelos desde el servicio de inferencia.

El sistema resultante permite procesar eventos de red y devolver predicciones en tiempo real, facilitando su integración en infraestructuras industriales IIoT donde la detección temprana de anomalías resulta crítica para la seguridad y continuidad operativa.

Para ello, se plantean los siguientes objetivos:

- Exponer los modelos entrenados mediante una API REST para permitir su consumo por sistemas externos.
- Separar claramente el entrenamiento offline del servicio de inferencia, evitando interferencias entre cargas.
- Garantizar la reproducibilidad del sistema mediante contenerización con Docker y orquestación con Docker Compose.
- Incorporar mecanismos de versionado, trazabilidad y selección dinámica del modelo activo.

4.2 Arquitectura y estructura del sistema

El sistema se diseña siguiendo una arquitectura modular orientada a producción, donde se separan claramente las distintas fases del ciclo de vida de un modelo de aprendizaje automático: ingesta de datos, preprocesamiento, entrenamiento, almacenamiento de artefactos e inferencia.

Esta arquitectura da continuidad a las decisiones tomadas en los hitos anteriores. En particular, el pipeline de transformación definido en el Hito 2 y el proceso de modelado del Hito 3 deben mantenerse alineados entre entrenamiento e inferencia, evitando inconsistencias entre los datos usados para entrenar y los datos recibidos durante la inferencia.

El flujo general del sistema puede describirse como:

1. Ingesta de datos y aplicación del pipeline de preprocesamiento.
2. Entrenamiento de nuevas versiones del modelo y generación de artefactos.
3. Almacenamiento versionado de modelos, métricas y metadatos.
4. Servicio de inferencia accesible mediante API REST.

La estructura del proyecto refleja esta separación de responsabilidades y facilita la trazabilidad, reproducibilidad y mantenimiento del sistema:

- `src/`: contiene la implementación completa del sistema, incluyendo los módulos de ingesta, preprocesamiento, entrenamiento, inferencia y API. Esta organización permite reutilizar el mismo pipeline en las distintas fases del sistema.
- `data_output/`: almacena los datos transformados y los artefactos de preprocesamiento necesarios para reproducir en inferencia las mismas transformaciones aplicadas durante el entrenamiento.
- `models_output/`: almacena los modelos entrenados junto con sus métricas, metadatos y codificadores de etiquetas asociados a cada versión. Esta estructura permite gestionar múltiples versiones de modelos y seleccionar dinámicamente el modelo activo en el sistema de inferencia.
- `examples/`: contiene ejemplos de entrada en formato JSON para validar el funcionamiento del endpoint de predicción y facilitar pruebas del sistema.
- `DockerFile.train`, `DockerFile.inferencia` y `docker-compose.yml`: definen la infraestructura de despliegue contenerizada, permitiendo la ejecución reproducible y la separación de servicios entre entrenamiento e inferencia.

4.3 Entrenamiento offline e inferencia

El sistema separa explícitamente las fases de entrenamiento e inferencia, siguiendo buenas prácticas en el despliegue de sistemas de aprendizaje automático.

El entrenamiento se ejecuta como un proceso offline mediante el servicio `train`. Este servicio realiza la ingesta del conjunto de datos, aplica el pipeline de preprocesamiento definido en el Hito 2 y entrena una nueva versión del modelo según la configuración definida. Como resultado, se generan los artefactos necesarios para su uso posterior en inferencia, incluyendo el modelo serializado, los artefactos de preprocesamiento, las métricas, los metadatos y el codificador de etiquetas.

Dado el tamaño del conjunto de datos y el coste computacional asociado al entrenamiento, esta fase se desacopla completamente del servicio de inferencia. De este modo, se evita que tareas intensivas en recursos afecten a la disponibilidad o latencia del sistema en tiempo de ejecución.

Por otro lado, la inferencia se implementa como un servicio independiente (`inferencia`), encargado de cargar dinámicamente el modelo activo y responder a peticiones en tiempo real mediante una API REST.

Esta separación aporta varias ventajas clave:

- Permite escalar el servicio de inferencia de forma independiente.
- Facilita la actualización o sustitución de modelos sin reconstruir todo el sistema.
- Evita la interferencia entre cargas de entrenamiento e inferencia.
- Mejora la estabilidad y disponibilidad del sistema en entornos productivos.

Ambos servicios comparten artefactos mediante volúmenes persistentes. De este modo, el servicio de inferencia utiliza el modelo, los metadatos y los transformadores generados durante el entrenamiento, manteniendo la coherencia del pipeline completo.

4.4 Servicio de inferencia y API REST

4.4.1 Diseño de la API

La exposición del modelo para su consumo por sistemas externos se ha resuelto mediante una API REST orientada a una arquitectura de microservicios. El servicio de inferencia se ha construido utilizando el *framework* **FastAPI**, seleccionado por su rendimiento, su integración con esquemas de validación de datos

y su capacidad para generar documentación interactiva automáticamente mediante Swagger UI y ReDoc.

A nivel arquitectónico, se ha implementado un patrón de inicialización tipo *Singleton* o de carga temprana (*Eager Loading*). En el archivo `src/api.py`, el modelo serializado, los artefactos de preprocesamiento y el codificador de etiquetas asociado se cargan en memoria durante el arranque del servidor. Esto reduce la latencia asociada a las operaciones de lectura en disco (I/O) que se produciría si se cargaran en cada petición individual, mejorando el tiempo de respuesta del servicio de inferencia.

Para dotar de robustez al sistema frente a paquetes de red anómalos o peticiones mal formadas, se ha integrado la librería **Pydantic**. Mediante la definición de esquemas estrictos de validación de datos, Pydantic asegura que cualquier paquete JSON de entrada (simulando un evento IIoT) contenga los tipos de datos correctos antes de pasarlos al motor de inferencia. Si el formato es inválido, FastAPI detiene la petición y devuelve automáticamente un error de validación HTTP 422, protegiendo al contenedor de entradas incompletas o mal formadas.

4.4.2 Endpoints disponibles

La API expone los siguientes puntos de acceso (*endpoints*) principales, diseñados para cubrir tanto la monitorización del sistema como la operación funcional y la gestión del modelo desplegado:

- **/health (GET):** endpoint de monitorización diseñado para que herramientas de orquestación, como Docker Compose o un futuro clúster de Kubernetes, comprueben el estado de salud del contenedor. Verifica que el servidor está activo, que el modelo se ha instanciado correctamente en memoria y que el codificador de etiquetas está disponible.
- **/model-info (GET):** devuelve información del modelo activo, incluyendo sus metadatos, la ruta del modelo cargado y la ruta de los artefactos de preprocesamiento utilizados.
- **/models (GET):** lista las versiones de modelos disponibles en el directorio de modelos y muestra cuál se encuentra activo.
- **/models/compare (GET):** permite comparar las métricas principales de las distintas versiones disponibles, como *accuracy*, *F1 macro* y *F1 weighted*.
- **/predict (POST):** endpoint principal de inferencia. Recibe una lista de eventos de red en formato JSON, aplica el preprocesamiento correspondiente y devuelve la clase predicha en formato legible junto con su codificación interna.
- **/admin/models/select (POST):** endpoint administrativo que permite seleccionar dinámicamente una versión concreta del modelo sin reiniciar el contenedor. Antes de activar el modelo, el sistema valida que el archivo exista y que pueda cargarse correctamente en memoria.

- `/admin/models/history (GET)`: devuelve el histórico de cambios de modelo registrados por el sistema.

La documentación interactiva de estos endpoints está disponible en:

`http://localhost:18080/docs`

4.4.3 Proceso de inferencia

El ciclo de vida de una petición de inferencia sigue un flujo controlado para reducir el riesgo de *training-serving skew*, es decir, discrepancias entre las transformaciones aplicadas durante el entrenamiento y las aplicadas durante la inferencia.

Al recibir un JSON válido en el endpoint `/predict`, el sistema convierte los registros de entrada en un `DataFrame` y adapta algunos nombres de columnas para mantener la compatibilidad con los nombres originales del conjunto de datos. A continuación, aplica los transformadores guardados durante el entrenamiento, como el imputador, el escalador, el codificador de variables categóricas y la selección final de columnas, cargándolos directamente desde los artefactos almacenados en `data_output/`.

Una vez que el vector de datos ha sido transformado al mismo espacio dimensional y escalar que el modelo espera, se ejecuta la inferencia sobre el modelo activo cargado en memoria. Como el modelo trabaja internamente con clases codificadas numéricamente, el servicio utiliza el `LabelEncoder` asociado a la versión activa para devolver una etiqueta interpretable.

Por ejemplo, una respuesta válida del endpoint es:

```
{  
  
  "prediccion": ["normal"],  
  
  "prediccion_codificada": [14]  
}
```

4.5 Contenerización y orquestación

Para garantizar la portabilidad del sistema, resolver el problema de dependencias y preparar la arquitectura para infraestructuras escalables, todo el ciclo de vida del aprendizaje automático se ha contenerizado utilizando **Docker**.

Se han diseñado dos imágenes independientes con responsabilidades atómicas:

- `DockerFile.train`: define un contenedor de ejecución efímera (*job*). Su propósito es instalar las dependencias necesarias, ejecutar secuencialmente el pipeline de ingesta, preprocesamiento y entrenamiento, generar los modelos y finalizar su ejecución.
- `DockerFile.inferencia`: define un contenedor persistente (*service*), encargado de servir la API mediante un servidor web ASGI con Uvicorn, manteniendo el puerto interno 8000 a la escucha. En la ejecución local mediante Docker Compose, este puerto se publica en el host como 18080.

La orquestación de estos contenedores y la comunicación entre sus fases se gestiona mediante el archivo `docker-compose.yml`. La decisión arquitectónica clave en esta orquestación ha sido el uso de **volúmenes compartidos**. En el manifiesto de *compose*, se han mapeado las rutas locales `./models_output` y `./data_output` hacia los directorios internos `/app/modelos` y `/app/datos/preprocesados` de ambos contenedores.

Este enfoque permite que, al finalizar el contenedor de entrenamiento, los modelos y artefactos recién generados se persistan en el volumen. Inmediatamente, el contenedor de inferencia tiene acceso a estos nuevos ficheros, permitiendo incorporar nuevos artefactos sin reconstruir las imágenes y activar una versión concreta mediante el endpoint administrativo de selección de modelos.

Además, tanto el servicio de inferencia como el de entrenamiento se configuran mediante variables de entorno. En el caso de inferencia, estas variables permiten definir las rutas del modelo activo, los artefactos de preprocesamiento, los metadatos y el histórico de cambios sin modificar el código fuente. En el caso del entrenamiento, permiten parametrizar la versión del modelo y algunos hiperparámetros, facilitando la generación de nuevas versiones con distintas configuraciones.

Finalmente, el archivo `docker-compose.yml` incorpora un *healthcheck* que consulta periódicamente el endpoint `/health`, permitiendo verificar automáticamente el estado del contenedor de inferencia.

4.6 Gestión y versionado de modelos

Para dotar al sistema de capacidades dinámicas y sentar las bases hacia una plataforma de operaciones de aprendizaje automático (MLOps), se ha implementado un mecanismo de gestión estructurada de los mod-

elos generados.

Durante la fase de entrenamiento offline, el script de modelado no se limita a exportar el objeto binario `.joblib`. Simultáneamente, el sistema serializa archivos de metadatos y métricas en formato `.json`. Estos archivos actúan como una firma del experimento e incluyen información crítica para auditorías: el *timestamp* de creación, las métricas de rendimiento en validación (*accuracy*, *F1 macro* y *F1 weighted*) y los hiperparámetros exactos utilizados.

En la implementación actual, los modelos se conservan mediante identificadores de versión, por ejemplo `modelo_v1.joblib` o `modelo_v2.joblib`, junto con un alias estable que representa el modelo utilizado por defecto por la API. Cada versión dispone además de sus propios archivos de metadatos, métricas y codificador de etiquetas, lo que permite comparar versiones, seleccionar el modelo activo y mantener trazabilidad sobre los artefactos desplegados.

El servicio de inferencia está diseñado para trabajar con esta estructura de versionado. Al arrancar, carga el modelo activo y los metadatos asociados a partir de la configuración definida para el contenedor. Además, mediante el endpoint `/admin/models/select`, el sistema puede seleccionar dinámicamente una versión concreta del modelo, siempre que el archivo exista y pueda cargarse correctamente en memoria.

Cada intento de cambio de modelo queda registrado en el archivo `model_selection_history.jsonl`, incluyendo la fecha, el modelo anterior, el nuevo modelo solicitado, el estado de la operación y el detalle del resultado. Este histórico puede consultarse mediante el endpoint `/admin/models/history`, aportando trazabilidad al proceso de promoción o retroceso de modelos.

4.7 Validación y pruebas del sistema

Para asegurar que el despliegue es funcional, se han realizado pruebas sobre el flujo completo de despliegue.

Entre las pruebas realizadas se incluye la construcción de las imágenes con `docker compose build`, la ejecución del entrenamiento offline con `docker compose run -rm train` y el arranque del servicio de inferencia con `docker compose up inferencia`.

La validación se ha apoyado en la interfaz interactiva Swagger UI, expuesta en la ruta `/docs`. A través de ella, se han inyectado los esquemas JSON de prueba alojados en el directorio `examples/` (como el fichero `predict_example.json`), los cuales contienen representaciones de tráfico de red para comprobar el funcionamiento del endpoint de predicción.

Los resultados de esta validación demuestran cuatro aspectos fundamentales:

1. **Fiabilidad del pipeline:** los eventos simulados superan los esquemas de validación, son correctamente transformados por los artefactos precalculados y arrojan predicciones consistentes con el Hito 3.
2. **Desempeño y latencia:** gracias a la arquitectura orientada a la carga del modelo en memoria, la API evita leer el modelo desde disco en cada petición, reduciendo el tiempo de respuesta del endpoint de inferencia.
3. **Aislamiento de responsabilidades:** la separación entre el contenedor de entrenamiento y el contenedor de inferencia evita mezclar tareas intensivas de entrenamiento con peticiones HTTP del servicio online.
4. **Gestión del modelo activo:** se ha comprobado la consulta de modelos disponibles, la comparación de métricas entre versiones y el registro de cambios de modelo mediante los endpoints administrativos.

4.8 Limitaciones, mejoras futuras y conclusiones

4.8.1 Limitaciones técnicas

A pesar del éxito en la operacionalización de la arquitectura, el sistema en su estado actual presenta ciertas limitaciones propias de los despliegues basados en Docker Compose. La escalabilidad está restringida a la capacidad vertical de un único nodo (*host*). Además, la API RESTful no implementa en esta iteración capas de seguridad profunda, como terminación TLS/HTTPS o autenticación de clientes mediante tokens JWT, asumiendo por el momento que operará detrás del firewall de una red OT privada.

Asimismo, el sistema no incorpora todavía monitorización continua de métricas operativas, como latencia, volumen de peticiones, tasa de errores o degradación del rendimiento del modelo en producción.

El almacenamiento de modelos se basa en ficheros locales compartidos mediante volúmenes Docker, por lo que en escenarios distribuidos sería necesario sustituirlo por un almacenamiento centralizado o un registro de modelos.

4.8.2 Mejoras futuras y visión MLOps

El diseño arquitectónico adoptado sienta una base para evolucionar hacia una plataforma MLOps de mayor madurez. Como siguientes pasos, se propone:

- Sustituir el versionado de ficheros locales por un *Model Registry* formal, integrando herramientas como **MLflow**.
- Migrar la persistencia de volúmenes Docker a un almacenamiento distribuido de objetos, como **MinIO** o AWS S3, permitiendo la propagación de modelos a múltiples fábricas o nodos perimetrales (*Edge*).
- Sustituir Docker Compose por **Kubernetes** para gestionar el contenedor de inferencia, habilitando el autoescalado horizontal real basado en el tráfico de red, así como políticas avanzadas de despliegue tipo *Canary* o *Blue/Green*.
- Proteger los endpoints administrativos mediante autenticación, autorización y auditoría más completa.
- Añadir tests automáticos para validar la API, el preprocesamiento y la selección de modelos.

4.8.3 Conclusiones

En este cuarto hito se ha llevado el modelo desarrollado en las fases anteriores a un entorno de despliegue funcional. El sistema permite realizar predicciones sobre eventos de red mediante una API REST, separando el entrenamiento offline del servicio de inferencia en tiempo real.

La arquitectura implementada permite reutilizar el mismo pipeline de preprocesamiento en entrenamiento e inferencia, reduciendo el riesgo de inconsistencias entre ambos procesos. Además, el uso de Docker y Docker Compose facilita una ejecución reproducible y separa claramente las responsabilidades de cada servicio.

Por último, la gestión versionada de modelos, métricas, metadatos y codificadores permite consultar las versiones disponibles, comparar su rendimiento y seleccionar dinámicamente el modelo activo. En conjunto, este hito deja una solución desplegable, trazable y preparada para evolucionar hacia una arquitectura MLOps más completa.

Hito 5: Monitorización y ciclo de retroalimentación

5.1 Introducción y objetivos

El despliegue de un modelo de aprendizaje automático en producción (alcanzado en el Hito 4) no representa el final del ciclo de vida del software, sino el comienzo de su fase operativa. Históricamente, los modelos de Inteligencia Artificial se trataban como artefactos estáticos; sin embargo, en entornos dinámicos e hiperconectados como las redes industriales IIoT, los datos cambian constantemente debido a la incorporación de nuevos sensores, actualizaciones de protocolos o la evolución continua de las tácticas de los atacantes cibernéticos.

El objetivo de este quinto y último hito es dotar al sistema de detección de intrusiones de capacidades avanzadas de observabilidad, monitorización y adaptación continua, abrazando por completo el paradigma **MLOps** (Machine Learning Operations). Se persigue transformar un servicio de inferencia pasivo en una plataforma viva y resiliente, capaz de auditar su propio rendimiento, detectar señales de degradación, registrar evidencias, alertar al operador y facilitar un ciclo controlado de validación y reentrenamiento.

Para alcanzar esta madurez tecnológica, se plantean los siguientes objetivos específicos:

- Implementar una capa de observabilidad robusta basada en la pila **Prometheus** y **Grafana** para registrar y visualizar métricas operativas (DevOps) y del modelo (Data Science).
- Desarrollar un mecanismo algorítmico automatizado capaz de detectar la **deriva de datos** (*Data Drift*) mediante el análisis estadístico de ventanas temporales.
- Configurar un sistema de **alertas externas asíncronas** (vía API de Telegram) que notifique proactivamente a los operadores del Centro de Operaciones de Seguridad (SOC) cuando se superen umbrales críticos.
- Cerrar el ciclo de vida de los datos mediante un **ciclo de retroalimentación** que incluya una cola de validación humana y la ejecución de un pipeline de reentrenamiento y despliegue automatizado

(*Zero-Downtime Deployment*) de una nueva versión del modelo.

5.2 Arquitectura de monitorización

La arquitectura de monitorización se ha diseñado siguiendo prácticas habituales en arquitecturas basadas en microservicios contenerizados. Se ha optado por un modelo de extracción de telemetría (*pull-based telemetry*) orquestado mediante Docker Compose, compuesto por tres capas fundamentales aisladas y comunicadas mediante una red virtual interna:

1. **Capa de Instrumentación (API FastAPI):** El servicio de inferencia ha sido extendido mediante código para calcular sus propias estadísticas en la memoria RAM. Expone un endpoint dedicado y estandarizado (`/metrics`) que sirve como punto de recolección en tiempo real.
2. **Motor de Recolección (Prometheus):** Desplegado como un contenedor independiente, se ha configurado mediante el archivo `prometheus.yml` para que actúe como un *scraper*. Cada pocos segundos, realiza peticiones HTTP GET al endpoint de la API, extrae el texto plano con los valores numéricos y los indexa en su base de datos interna orientada a series temporales (TSDB).
3. **Capa de Visualización (Grafana):** Un servicio web conectado por *backend* a la base de datos de Prometheus. A través del aprovisionamiento automático, Grafana renderiza cuadros de mando (*dashboards*) dinámicos ejecutando consultas en el lenguaje PromQL, permitiendo a los ingenieros observar el pulso de la planta de manera visual e interactiva.

Junto a esta pila tecnológica *open-source*, se han acoplado módulos independientes en el código fuente de Python (`src/drift.py` y `src/alerting.py`) que encapsulan la lógica de negocio para la monitorización matemática y el enrutamiento de las notificaciones.

5.3 Instrumentación de la API

Para que Prometheus pueda recolectar información, ha sido imperativo instrumentar el código del servidor REST. Se ha integrado la librería oficial `prometheus-client` en el núcleo de FastAPI, inyectando un *middleware* asíncrono que intercepta todo el tráfico de entrada y salida de red.

Esta instrumentación se ha diseñado para ser de muy baja latencia, garantizando que medir el comportamiento del contenedor no afecte negativamente al tiempo real de inferencia. Para ello, se han definido diferentes estructuras de datos en memoria:

- **Counters (Contadores):** Variables de incremento monotónico. Se utilizan para medir el tráfico absoluto (ej. `http_requests_total`).

- **Histograms (Histogramas):** Agrupan valores en *buckets* o intervalos. Son esenciales para calcular la distribución de la latencia (`request_duration_seconds`), permitiendo aislar peticiones anómalas muy lentas.
- **Gauges (Indicadores):** Variables que pueden subir y bajar libremente. Se emplean para medir estados actuales del sistema, como el tamaño de la cola de revisión de datos anómalos, el modelo activo o el estado del reentrenamiento.

Cada vez que el endpoint `/predict` procesa un paquete de red, estas estructuras actualizan sus valores en hilos de ejecución paralelos. Cuando Prometheus interroga al servidor, la API vuelca estas estructuras en un formato de texto estandarizado.

5.4 Métricas operativas del servicio (DevOps)

La primera dimensión de la observabilidad corresponde a la salud y resiliencia de la infraestructura de software. En el contexto industrial, un modelo predictivo con una precisión perfecta carece de utilidad operativa si el microservicio que lo aloja está caído o severamente saturado. Por ello, se han implementado las siguientes métricas operativas críticas:

- **Tasa de Peticiones (RPS - Requests Per Second):** Monitoriza el volumen de tráfico IIoT bruto que está siendo analizado. Una caída abrupta a cero indica un fallo en la recolección de los sensores, mientras que un pico exponencial puede evidenciar un ataque de denegación de servicio (DDoS) contra la propia API.
- **Latencia de Inferencia (Percentiles P95 y P99):** Mide el tiempo de respuesta del servidor en milisegundos. Dado que el sistema debe mitigar ataques antes de que afecten a los PLCs de la fábrica, garantizar una latencia inferior a los umbrales de seguridad es crítico.
- **Métricas técnicas del servicio:** Supervisión del estado observable de la API, complementable en un entorno productivo con métricas específicas del contenedor, como CPU, RAM o uso de disco. En esta iteración, el foco se sitúa en las métricas expuestas por la propia aplicación y consumidas por Prometheus.
- **Tasa de Errores HTTP:** Contabilización segmentada de códigos de error. Se discriminan los errores del cliente 4xx (como el código HTTP 422 devuelto por la librería Pydantic cuando un paquete JSON carece de variables necesarias) de los errores 5xx (excepciones internas y caídas del servidor Python).

5.5 Logging y trazabilidad de inferencias

Para posibilitar el análisis asíncrono y la detección de derivas complejas, el sistema requiere una "memoria" persistente de sus propias decisiones. Se ha implementado un mecanismo avanzado de *logging* en el que

cada predicción no solo se devuelve al cliente, sino que se almacena de forma estructurada en un archivo de registros continuos (.jsonl).

Este archivo serializa las características extraídas del paquete de red original junto con la etiqueta predicha, su codificación interna, la versión del modelo activo y el estado de deriva asociado a la muestra.

Esta trazabilidad (*Ground Truth diferido*) es el pilar fundacional de la validación. En el ámbito de la ciberseguridad, raramente se tiene certeza absoluta de si un evento fue un ciberataque real hasta que un analista humano lo investiga *a posteriori*. Guardar el contexto íntegro de la inferencia permite cruzar posteriormente la decisión automática del algoritmo con el veredicto real del operador.

5.6 Monitorización de predicciones (Model Health)

A diferencia de las métricas de infraestructura (CPU, RAM), las métricas del modelo evalúan el comportamiento interno de la Inteligencia Artificial. La mayor complejidad en la monitorización en producción radica en la ausencia de la etiqueta real (*Ground Truth*) durante la fase de inferencia. Al no disponer de esta etiqueta, es matemáticamente imposible calcular métricas de rendimiento estáticas tradicionales como el *Accuracy*, *Recall* o el *F1-Score* instantáneo.

Para solventar este obstáculo, se ha instrumentado la **distribución estadística de las predicciones**. A través de un contador multidimensional en Prometheus, se registra en tiempo real el porcentaje de tráfico que el modelo clasifica en cada una de sus categorías de salida (tráfico normal, *mitm*, *fuzzing*, *dos*, etc.). Si empíricamente el 85% del tráfico de la planta es históricamente legítimo, y el *dashboard* evidencia de forma súbita que el modelo está clasificando el 90% del tráfico como malicioso, los operadores de seguridad (SOC) son conscientes inmediatamente de dos escenarios posibles: la planta está bajo un ataque masivo sin precedentes, o el modelo predictivo ha colapsado y está generando una avalancha de falsos positivos debido a cambios en la red.

5.7 Detección de deriva de datos (Data Drift)

El núcleo analítico más complejo de este hito reside en la implementación algorítmica del módulo `src/drift.py`. La deriva de datos (*Data Drift*) es el fenómeno mediante el cual la distribución estadística y probabilística de las variables de entrada (*features*) en producción difiere significativamente del espacio de estado con el que se entrenó el modelo original (X-IIoTID `dataset.csv`).

El script de deriva diseñado actúa como un vigilante en segundo plano. Su funcionamiento consiste en comparar las muestras recientes contra un perfil estadístico de referencia construido a partir del tráfico

normal de validación. Para cada muestra se calcula qué proporción de variables queda fuera de los rangos esperados y, posteriormente, esta información se agrega sobre una ventana deslizante.

Si las variables críticas de red (por ejemplo, el ratio de bytes transmitidos por segundo o la longitud media de los paquetes) sufren mutaciones radicales—ya sea porque la fábrica ha modernizado sus equipos o porque los atacantes han ofuscado su malware—el script supera su umbral de tolerancia y levanta un *flag* interno de advertencia. Este análisis exhaustivo blindo al sistema contra el *Concept Drift*, impidiendo que el algoritmo continúe emitiendo veredictos a ciegas en un entorno topológico que ya no comprende ni representa.

5.8 Sistema de alertas (Push Notifications)

La observabilidad basada exclusivamente en cuadros de mando visuales (*dashboards*) se considera pasiva y resulta insuficiente para la gestión de incidentes críticos, ya que requiere que un humano mire constantemente la pantalla. Para facilitar una respuesta temprana ante incidentes, se ha desarrollado el módulo `src/alerting.py`, diseñado para orquestar notificaciones externas asíncronas de tipo *push*.

Se ha integrado un sistema de mensajería comercial robusto basado en la API de **Telegram**. La instancia de este servicio se realiza de forma segura, inyectando un token de acceso criptográfico y un identificador de sala (*Chat ID*) a través de variables de entorno (`.env`), aislando así las credenciales del código fuente.

El servicio evalúa en bucle un conjunto de reglas lógicas predefinidas. Si se constata que el módulo de deriva detecta anomalías severas, o si las métricas de Prometheus revelan que la latencia operativa supera un percentil crítico (ej. 500 ms), el sistema redacta una carga útil de advertencia y dispara automáticamente una notificación detallada al dispositivo móvil del administrador de guardia, adjuntando la naturaleza del incidente para agilizar la fase de *triage*.

Durante la validación del sistema se comprobó este canal mediante alertas de prueba y alertas asociadas a condiciones anómalas del servicio, verificando que la notificación llegaba correctamente al operador. En la **figura D.1** del **Anexo D** puede observarse los ejemplos de alertas emergentes por el chat de Telegram.

5.9 Dashboard de Grafana

Toda la telemetría distribuida converge y se unifica en un panel visual centralizado en Grafana. Para asegurar la reproducibilidad del proyecto entre distintos entornos (desarrollo, pre-producción y producción), el *dashboard* se ha definido como código (IaC) e instanciado automáticamente al arrancar el contenedor a

través del fichero de provisión `disia-api-observability.json`.

El panel de control ha sido estructurado ergonómicamente en dos grandes dominios de responsabilidad:

1. **Salud del Servicio (Ingeniería de Plataforma):** Paneles superiores compuestos por gráficos de series temporales que trazan la carga transaccional (RPS), la saturación de los recursos de hardware y la proporción de códigos HTTP fallidos.
2. **Salud del Modelo (Data Science y SOC):** Paneles inferiores que exponen, mediante gráficos de anillos y barras apiladas, la distribución volumétrica de las predicciones. Destaca especialmente el medidor (*Gauge*) que expone en tiempo real el tamaño de la Cola de Revisión de anomalías.

En la prueba con tráfico anómalo se observó el aumento de la cola de revisión y el cambio en las métricas de deriva, lo que permitió comprobar visualmente que Prometheus y Grafana estaban recogiendo correctamente el estado del sistema. En la **figura D.2** del **Anexo D** puede observarse un ejemplo de visualización de Grafana.

5.10 Ciclo de retroalimentación y validación humana

El hallazgo de deriva estadística y la consecuente inyección de datos desconocidos en la Cola de Revisión desencadenan la necesidad imperiosa del ciclo de retroalimentación (*feedback loop*). Atendiendo a las mejores prácticas de ciberseguridad industrial, se ha diseñado un flujo de trabajo conservador de tipo *Human-in-the-Loop* (HITL), impidiendo que el sistema tome decisiones destructivas de auto-actualización de forma completamente autónoma.

Cuando se emite una alerta, el equipo de analistas de seguridad debe intervenir. Su tarea consiste en examinar las muestras recientes que el sistema detectó como fuera del perfil normal esperado y proporcionarles las etiquetas taxonómicas correctas (*Ground Truth*). Una vez que este nuevo conocimiento experto ha sido integrado en el almacén de datos validados y supera el umbral configurado por la variable de entorno `RETRAINING_MIN_VALIDATED_SAMPLES`, el sistema desbloquea y desencadena la política de reentrenamiento.

Para soportar este flujo, la API incorpora endpoints administrativos específicos para consultar la cola de revisión y registrar etiquetas validadas, concretamente `/admin/retraining/review-queue` y `/admin/retraining/lab`. De este modo, se separan explícitamente las predicciones automáticas del modelo de las etiquetas reales aportadas mediante revisión humana.

5.11 Política de reentrenamiento (*Retraining*)

El proceso de regeneración del modelo se ha implementado mediante el módulo `src/retraining.py`, adoptando en esta primera iteración una estrategia de reentrenamiento simple (*Simple Retraining* o *Stateless Retraining*).

Al activarse, el script ejecuta el pipeline de entrenamiento sobre los volúmenes compartidos de datos y modelos. El proceso incorpora el *dataset* original junto con las nuevas muestras validadas, actualiza los ficheros de entrenamiento necesarios y vuelve a ajustar el modelo Random Forest siguiendo el mismo flujo definido para el entrenamiento offline.

Como resultado de este proceso, el pipeline actualiza el artefacto del modelo reentrenado junto con sus métricas, metadatos y codificador de etiquetas. Estos ficheros se depositan en el volumen de almacenamiento persistente compartido `models_output/`.

5.12 Activación del nuevo modelo en producción

Cerrando de manera íntegra el ciclo operativo MLOps, la nueva versión del algoritmo debe ser promocionada al entorno de producción sin provocar ninguna disrupción en el escaneo de los paquetes de red, logrando un despliegue sin tiempo de inactividad (*Zero-Downtime Deployment*).

Haciendo uso de la versatilidad de la arquitectura dinámica de FastAPI, cuando el proceso de reentrenamiento finaliza correctamente la API recarga automáticamente el artefacto actualizado en memoria, actualiza sus metadatos y registra el cambio en el histórico de modelos. De este modo, se evita que el servicio continúe utilizando el objeto del modelo anterior que ya estaba cargado en RAM, y las nuevas peticiones de inferencia pasan a resolverse con el modelo reentrenado sin necesidad de reiniciar manualmente el contenedor.

5.13 Prueba completa del sistema (Validación End-to-End)

Para auditar y certificar la madurez técnica y la integración fluida de todo el ecosistema diseñado a lo largo de este proyecto, se ha ejecutado un protocolo de estrés haciendo uso del módulo `scripts/simulate_traffic.py`. Este *script* actúa como un inyector de carga que bombardea el servidor con peticiones HTTP que emulan el comportamiento de múltiples sensores IIoT.

Durante la demostración práctica documentada para esta memoria, el protocolo siguió las siguientes fases:

1. Se inyectó un volumen de 5000 peticiones de tráfico de red estandarizado, evidenciando un rendimiento estable en Grafana y una distribución normal del tráfico.
2. Se alteró deliberadamente el perfil matemático inyectando 500 vectores con características anómalas forzadas (simulando una alteración en la topología de la planta).
3. De manera automática, la plataforma reconoció la perturbación, el semáforo visual de Grafana registró el pico en la cola de revisión y el módulo de alertas envió la notificación urgente vía Telegram.
4. Tras simular la curación experta ejecutando `validate_review_queue.py`, el sistema detectó la superación del umbral volumétrico, ejecutando el pipeline de reentrenamiento en segundo plano y cargando la iteración mejorada en el núcleo de la API en vivo.

Los registros del contenedor de inferencia permitieron verificar la secuencia final del ciclo: detección de muestras suficientes, ejecución del proceso de reentrenamiento y recarga del modelo actualizado en memoria. En la **figura D.3** del **Anexo D** puede observarse un ejemplo de los logs del procedimiento de re-entrenamiento.

5.14 Limitaciones y mejoras futuras

Si bien el ciclo funcional diseñado cubre los elementos principales de una arquitectura MLOps aplicada al caso de estudio, la escala y rigidez de entornos de producción de misión crítica (como redes eléctricas o plantas petroquímicas) invitan a contemplar áreas de iteración futura:

- **Seguridad de endpoints administrativos:** En esta versión, los endpoints administrativos se mantienen sin autenticación al estar orientados a una prueba local controlada. En un entorno productivo deberían protegerse mediante autenticación, autorización y auditoría de operaciones.
- **Estrategias de Despliegue Avanzadas:** Actualmente, el sistema realiza una sobrescritura directa de los punteros en memoria (*Direct Replacement*). Para mitigar el riesgo de desplegar un modelo corrompido, sería recomendable implementar un esquema de **Shadow Deployment** (el nuevo modelo analiza tráfico real de manera silenciosa para comparar sus resultados con el modelo *legacy* antes de tomar el control total) o un esquema **Canary Release**.
- **Escalabilidad del Almacenamiento de Telemetría:** Prometheus ha sido instanciado con volúmenes de retención locales. En una infraestructura industrial que exige una retención de series temporales por periodos plurianuales (por motivos de auditoría y *compliance* normativo), sería imprescindible federar la arquitectura hacia soluciones de almacenamiento remoto en la nube como Thanos o Grafana Mimir.

- **Ergonomía en la Validación de Ground Truth:** El etiquetado manual a través de *scripts* de terminal presenta fricción operativa. Para facilitar el trabajo diario del *Security Operations Center* (SOC), se requeriría el desarrollo de una aplicación web (UI) que presentara las tramas de red anómalas de forma gráfica y amigable para acelerar el proceso de veredicto humano.

5.15 Conclusiones

La convergencia de herramientas nativas de la nube (*Cloud Native*) como Docker, Prometheus y Grafana, fusionadas simbióticamente con la capacidad algorítmica de evaluar su propia degradación estadística, y dotadas de la agilidad operativa para notificar incidentes vía Telegram y ejecutar ciclos de reentrenamiento no disruptivos, demuestra una comprensión integral del ciclo de vida del dato moderno.

Este sistema permite mantener bajo observación el servicio de inferencia y aporta mecanismos para que la Inteligencia Artificial desplegada sea más auditable, trazable y sostenible frente al paso del tiempo. Con ello, el proyecto consolida la ciberseguridad industrial predictiva no como una herramienta estática, sino como un componente operativo capaz de generar evidencias, alertas y ciclos controlados de mejora.



Directorio de GitHub

Para la mayor organización posible, todo el código realizado del proyecto se ha almacenado en un directorio de la plataforma GitHub. Accesible desde:

<https://github.com/Iru12/Proyecto-DISIA>

Análisis Exploratorio de los datos

En esta sección, recopilamos los distintos resultados obtenidos durante el hito 2 del proyecto. Se puede apreciar los distintos análisis realizados con la varianza y correlación de las características del conjunto de datos.

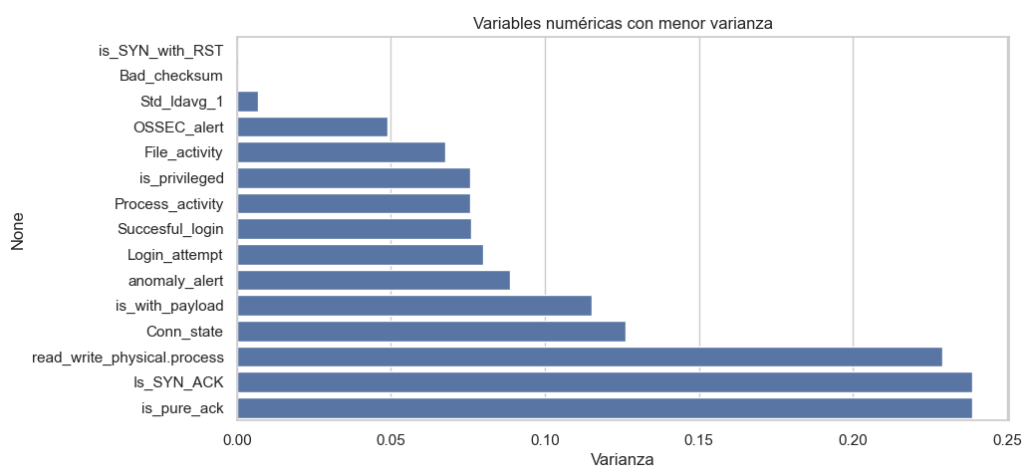


Figure B.1: Análisis de varianza de las características.

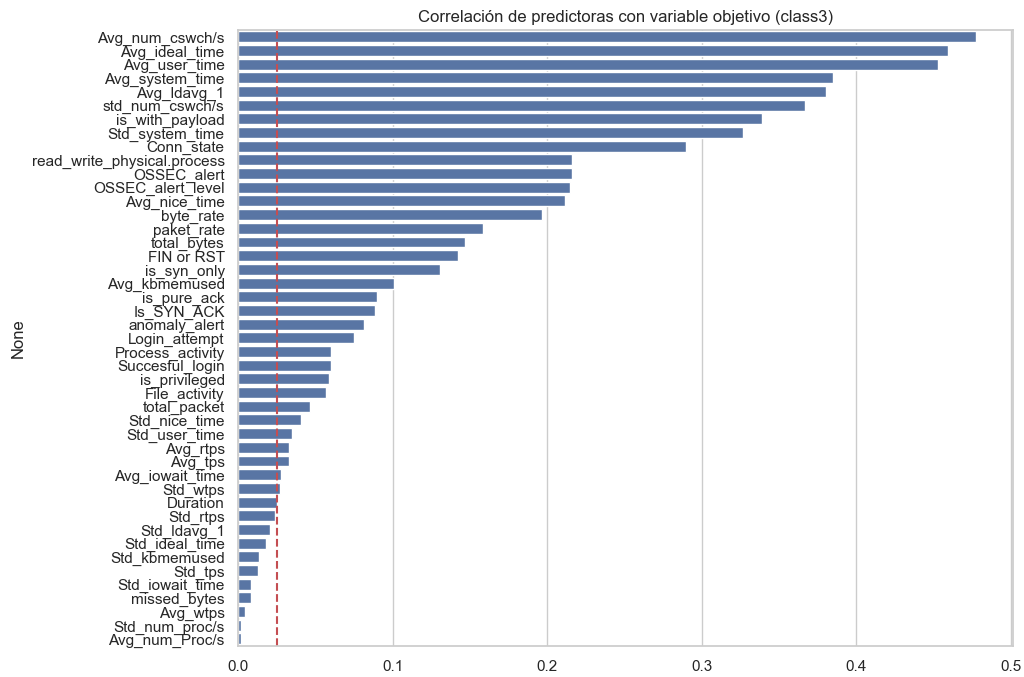


Figure B.2: Correlación de las variables con class3.

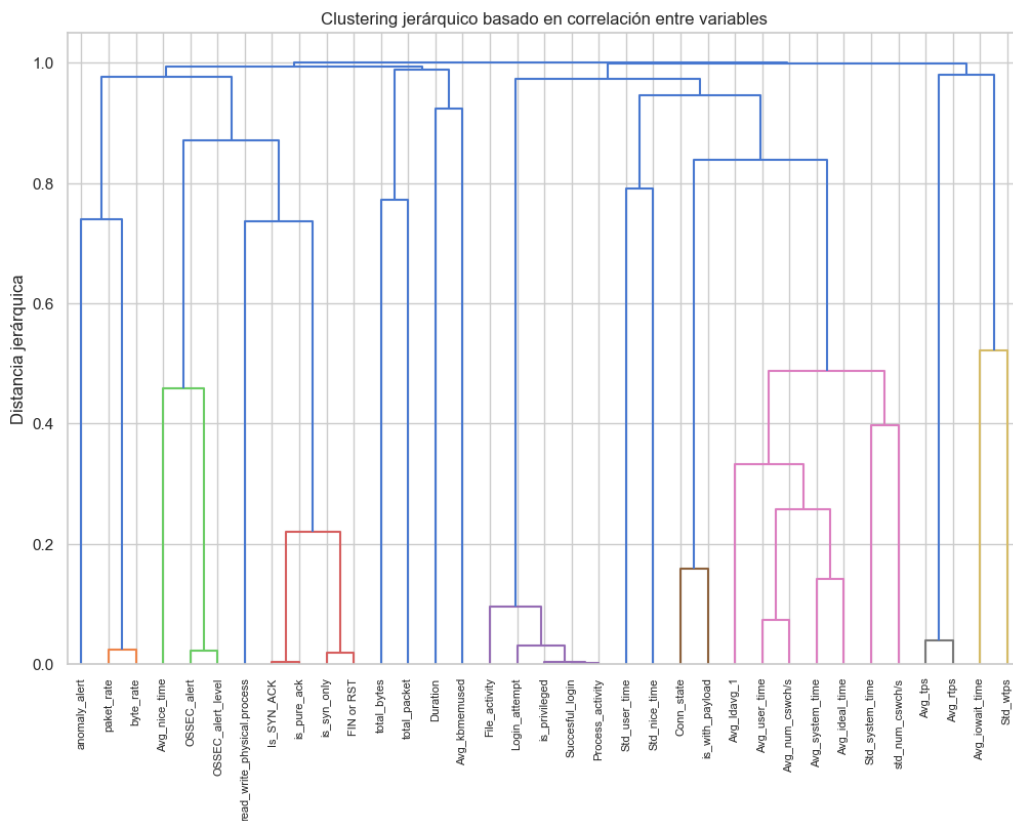


Figure B.3: Dendrograma de correlación entre características.

Visualizaciones de los resultados en entrenamiento

Para esta sección, recopilamos las gráficas y figuras obtenidas durante el desarrollo completo y explicación del hito 3.

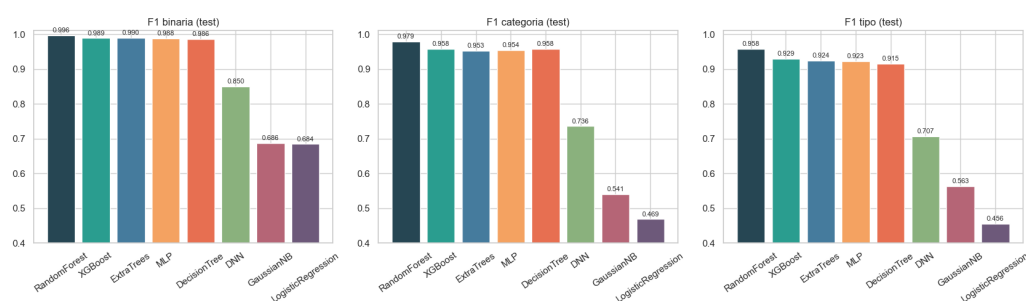


Figure C.1: Visualización de los resultados obtenidos de las métricas en test por cada modelo.

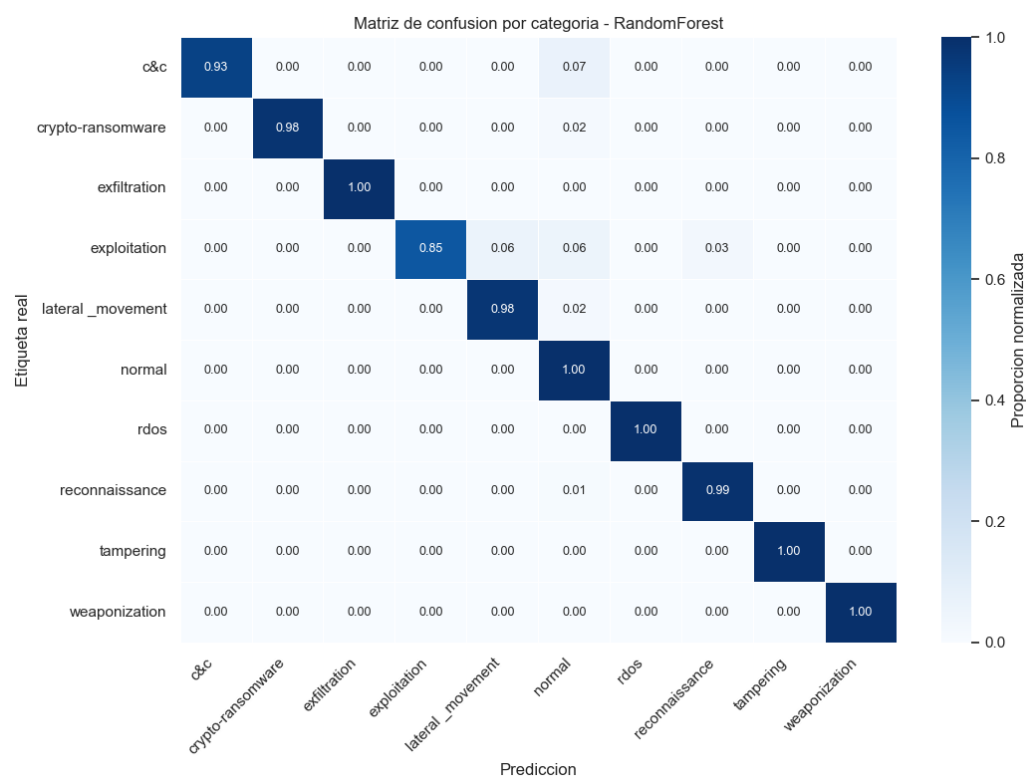


Figure C.2: Visualización de la matriz de confusión para el modelo RandomForest.

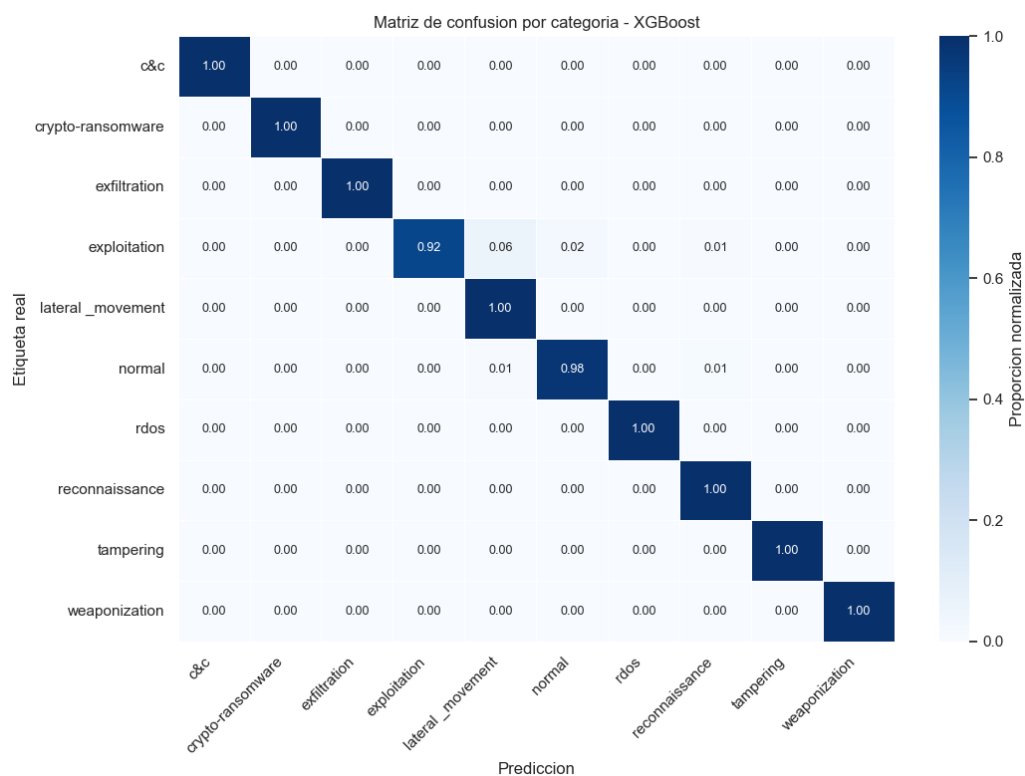


Figure C.3: Visualización de la matriz de confusión para el modelo XGBoost.



Figure C.4: Visualización de la matriz de confusión para el modelo ExtraTrees.

Visualización del proceso de monitorización

Para esta sección, recopilamos las fotos del proceso de despliegue y monitorización del hito 5.

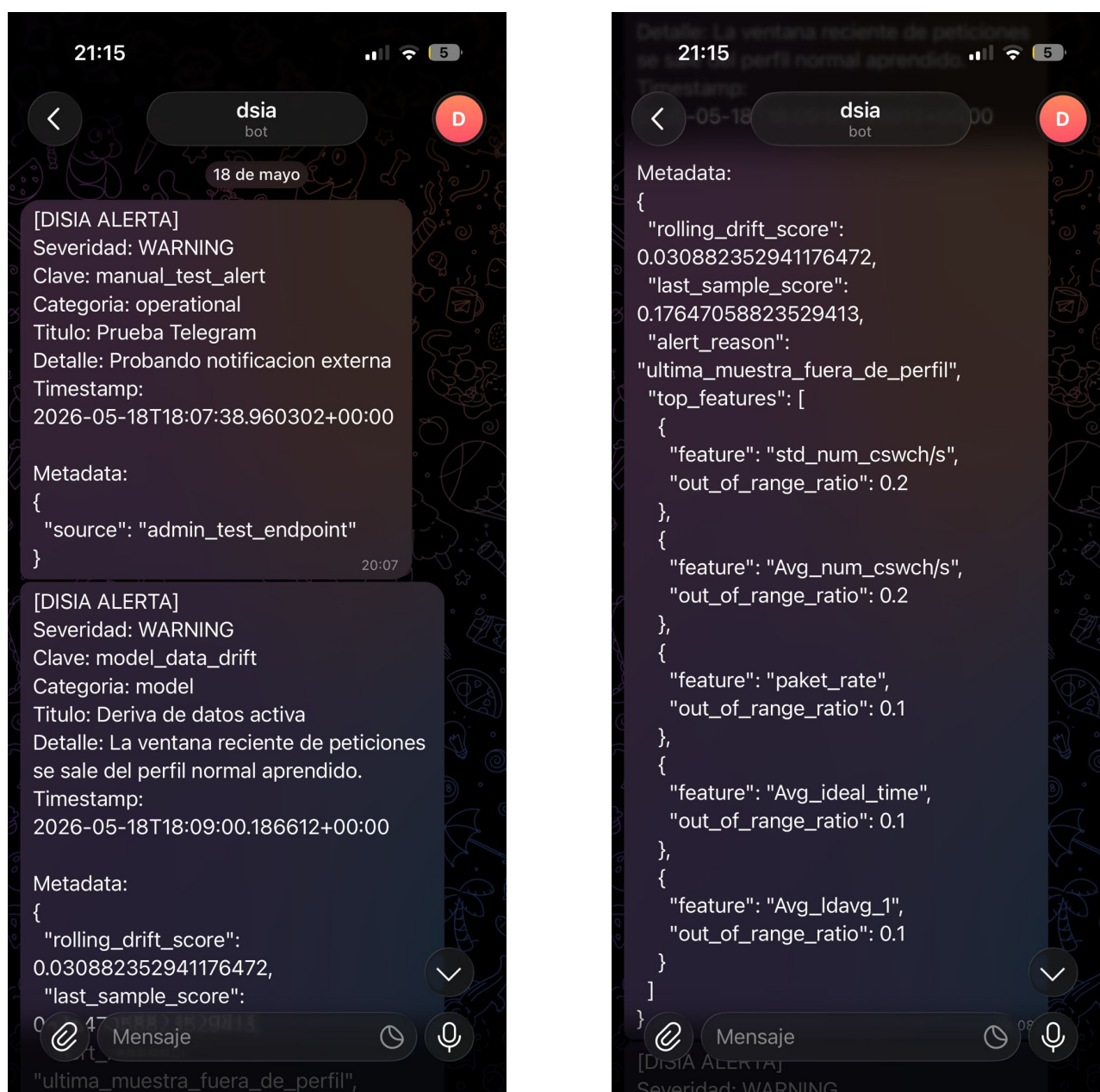


Figure D.1: Alertas recibidas en Telegram.

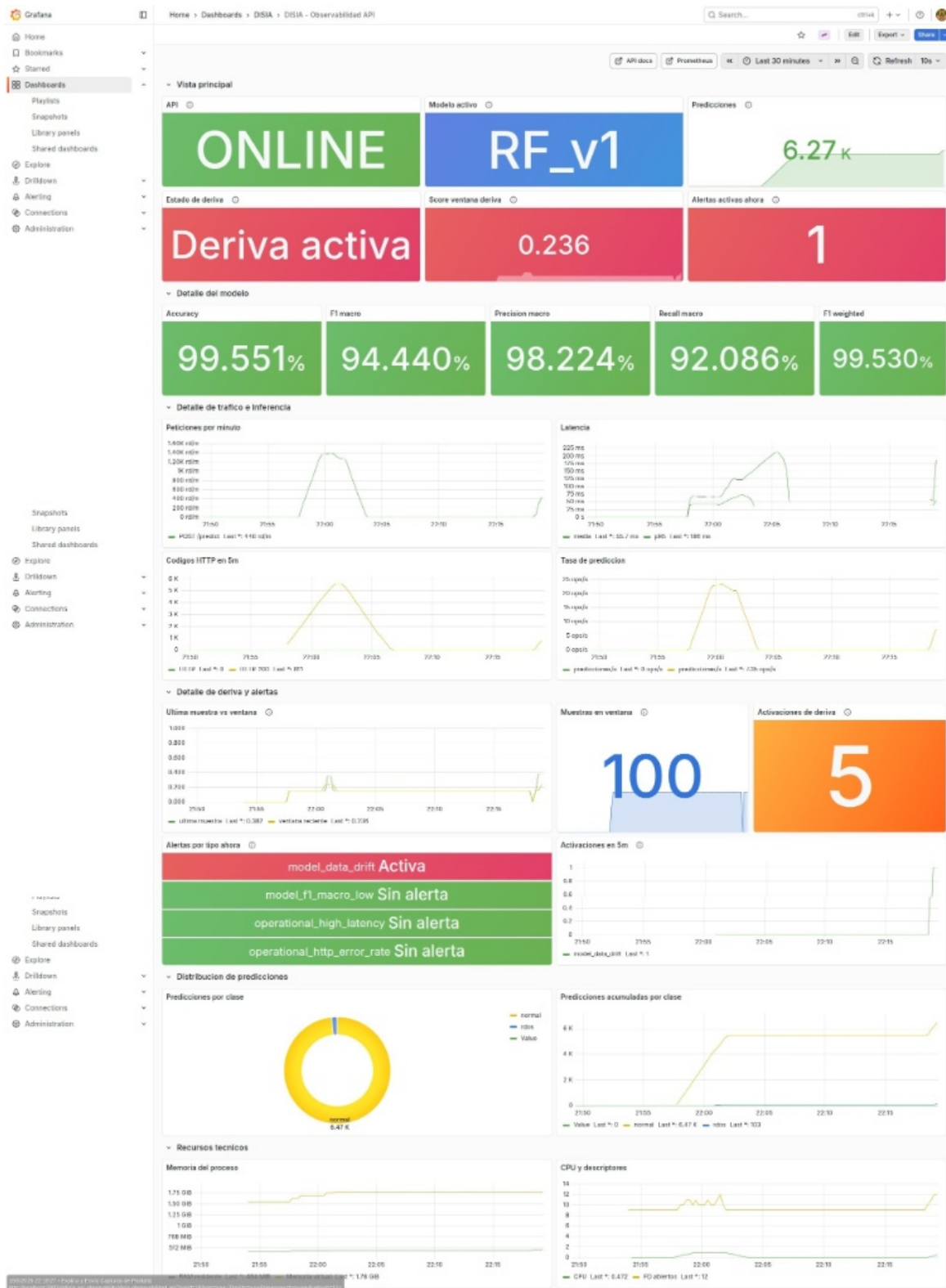


Figure D.2: Dashboard de monitorización en Grafana.

```

api | [14]
api | Re-entrenamiento ya en progreso. Se evaluará la necesidad de iniciar otro re-entrenamiento al finalizar el actual.
api | INFO: 172.18.0.1:33032 - "POST /predict HTTP/1.1" 200 OK
api | [14]
api | Re-entrenamiento ya en progreso. Se evaluará la necesidad de iniciar otro re-entrenamiento al finalizar el actual.
api | INFO: 172.18.0.1:33038 - "POST /predict HTTP/1.1" 200 OK
api | [14]
api | Re-entrenamiento ya en progreso. Se evaluará la necesidad de iniciar otro re-entrenamiento al finalizar el actual.
api | INFO: 172.18.0.1:33040 - "POST /predict HTTP/1.1" 200 OK
api | [14]
api | Re-entrenamiento ya en progreso. Se evaluará la necesidad de iniciar otro re-entrenamiento al finalizar el actual.
api | INFO: 172.18.0.1:33054 - "POST /predict HTTP/1.1" 200 OK
api | [14]
api | Re-entrenamiento ya en progreso. Se evaluará la necesidad de iniciar otro re-entrenamiento al finalizar el actual.
api | INFO: 172.18.0.1:33064 - "POST /predict HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:48334 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:58010 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:36052 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:38080 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:59222 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:35792 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:34200 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:56948 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:38690 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:60400 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:32900 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:51916 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:33508 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:48016 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:39574 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:46098 - "GET /metrics HTTP/1.1" 200 OK
api | INFO: 127.0.0.1:47478 - "GET /health HTTP/1.1" 200 OK
api | INFO: 172.18.0.4:52690 - "GET /metrics HTTP/1.1" 200 OK
api | Ruta de los datos preprocesados: /app/datos/preprocesados
api | Ruta donde se guardara el modelo entrenado: /app/modelos
api | Version del modelo: RF_v2
api | Iniciando el entrenamiento del modelo con los datos en: /app/datos/preprocesados
api | Entrenando Random Forest...
api | Random Forest entrenado en 144.13 segundos.
api | Entrenamiento completado exitosamente.
api | Modelo versionado guardado en: /app/modelos/RF_v2.joblib
api | Alias activo guardado en: /app/modelos/RF_model.joblib
api | Iniciando proceso de re-entrenamiento con etiquetas validadas: RF_v2...
api | Re-entrenamiento completado con 80 muestras validadas.
api | Etiquetas procesadas archivadas en: /app/datos/preprocesados/retraining_validated_processed_2026-05-19_21-05-14.jsonl
api | Modelo de detección de anomalías cargado exitosamente.
api | Modelo reentrenado cargado en memoria: RF_v2

```

Figure D.3: Logs del proceso de reentrenamiento.

Recursos Hardware

En esta sección, recopilamos los recursos hardware necesarios para la realización del proyecto.

Elemento	Recurso
Sistema Operativo	Windows-11-10.0.26200-SP0
CPU	AMD64 Family 25 Model 33 Stepping 2, AuthenticAMD
Arquitectura	AMD64
Núcleos Físicos	6
Núcleos Lógicos	12
RAM total (GB)	31.9300
CUDA Disponible	No
Disp. de entrenamiento	CPU

Table E.1: Recursos Hardware utilizados para el entrenamiento.

References

- [1] K. Schwab, *La cuarta revolución industrial*. Debate, 2016.
- [2] M. S. Rahman, T. Ghosh, N. Aurna, M. S. Kaiser, M. Anannya, and A. S. M. Hosen, “Machine learning and internet of things in industry 4.0: A review,” *Measurement: Sensors*, vol. 28, p. 100 822, Jun. 2023. DOI: 10.1016/j.measen.2023.100822.
- [3] B. Zhu, A. Joseph, and S. Sastry, “A taxonomy of cyber attacks on scada systems,” in *2011 International Conference on Internet of Things and 4th International Conference on Cyber, Physical and Social Computing*, 2011, pp. 380–388. DOI: 10.1109/iThings/CPSCoM.2011.34.
- [4] H. Boyes, B. Hallaq, J. Cunningham, and T. Watson, “The industrial internet of things (iiot): An analysis framework,” *Computers in Industry*, vol. 101, pp. 1–12, 2018, ISSN: 0166-3615. DOI: <https://doi.org/10.1016/j.compind.2018.04.015>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0166361517307285>.
- [5] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, *X-iiotid: A connectivity- and device-agnostic intrusion dataset for industrial internet of things*, 2021. DOI: 10.21227/mpb6-py55. [Online]. Available: <https://dx.doi.org/10.21227/mpb6-py55>.
- [6] AI/ML Programming, *Real time anomaly detection for industrial iot*, <https://aimlprogramming.com/services/real-time-anomaly-detection-for-industrial-iot/>, 2026.
- [7] AI/ML Programming, *Edge based anomaly detection for industrial iot*, <https://aimlprogramming.com/services/edge-based-anomaly-detection-for-industrial-iot/>, 2026.
- [8] D. Kadirvelu, *What is the true cost of hiring ai developers in 2026*, <https://raascloud.io/what-is--the-true-cost-of-hiring-ai-developers/>, Dec. 2025.
- [9] D. Thompson, *Operational downtime: How manufacturers are using predictive maintenance to combat staggering costs*, <https://www.automation.com/article/operational-downtime-manufacturers--predictive-maintenance-costs>, Nov. 2025.

- [10] M. Al-Hawawreh, E. Sitnikoba, and N. Aboutorab, *X-iiotid: A connectivity- and device-agnostic intrusion dataset for industrial internet of things*, IEEE DataPort, 2022. [Online]. Available: <https://ieee-dataport.org/documents/x-iiotid-connectivity-and-device-agnostic-intrusion-dataset-industrial-internet-things>.
- [11] D. R. Cox, *The Regression Analysis of Binary Sequences*. Journal of the Royal Statistical Society. Series B (Methodological), 20(2), 215–242., 1958. [Online]. Available: <http://www.jstor.org/stable/2983890>.
- [12] R. Duda, P. Hart, and D. G. Stork, "Pattern classification," *Wiley Interscience*, 2001.
- [13] S. Han, Q. Wu, and Y. Yang, "Machine learning for internet of things anomaly detection under low-quality data," *International Journal of Distributed Sensor Networks*, vol. 18, no. 10, 2022. DOI: 10.1177/15501329221133765.
- [14] J. B. Awotunde, S. O. Folorunso, A. L. Imoize, J. O. Odunuga, C.-C. Lee, C.-T. Li, and D.-T. Do, "An ensemble tree-based model for intrusion detection in industrial internet of things networks," *Applied Sciences*, vol. 13, no. 4, 2023. DOI: 10.3390/app13042479.
- [15] S. H. Rafique, A. Abdallah, N. S. Musa, and T. Murugan, "Machine learning and deep learning techniques for internet of things network anomaly detection—current research trends," *Sensors*, vol. 24, no. 6, 2024. DOI: <https://doi.org/10.3390/s24061968>.
- [16] J. Quinlan, "Induction of decision trees," *Machine Learning*, pp. 81–106, 1986. DOI: <https://doi.org/10.1007/BF00116251>.
- [17] B. Leo, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. DOI: <https://doi.org/10.1023/A:1010933404324>.
- [18] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. DOI: <https://doi.org/10.1037/h0042519>.
- [19] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. DOI: <https://doi.org/10.1038/323533a0>.
- [20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.
- [21] M. T. Ribeiro, S. Singh, and C. Guestrin, "' why should i trust you?' explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016, pp. 1135–1144.
- [22] H. Liang, X. Pei, X. Jia, W. Shen, and J. Zhang, "Fuzzing: State of the art," *IEEE Transactions on Reliability*, vol. 67, no. 3, pp. 1199–1218, 2018.

- [23] J. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986. DOI: <https://doi.org/10.1007/BF00116251>.